Practical No. 1

Python Warmup (NumPy)

Aim:

To understand and practice the basics of NumPy: creating arrays, reshaping, indexing and slicing, vectorized operations, random number generation and basic statistics.

Theory

NumPy (Numerical Python) is a popular Python library used for numerical computations and scientific programming. It provides a powerful multidimensional array object and various functions to perform mathematical operations efficiently.

NumPy is faster and more memory-efficient than normal Python lists when working with large amounts of numerical data. It is widely used in data analysis, machine learning, artificial intelligence, and scientific computing.

1. Creating Arrays

NumPy arrays can be created using the np.array() function. Arrays can be one-dimensional, two-dimensional, or multidimensional.

Example: np.array([10, 20, 30, 40])

2. Reshaping Arrays

Reshaping changes the dimensions of an array without changing its elements. The reshape() function is used for this purpose.

Example: A one-dimensional array containing 6 elements can be reshaped into a 2 × 3 matrix.

3. Indexing and Slicing

- Indexing: Accesses a specific element of an array using its position.
- Slicing: Extracts a portion of an array using a range of indices.

These operations help retrieve and manipulate specific data efficiently.

4. Vectorized Operations

Vectorization allows mathematical operations to be performed on entire arrays without using explicit loops. It makes code shorter, easier to read, and often faster.

Example: Adding 5 to every element of an array using arr + 5.

5. Random Number Generation

NumPy provides the np.random module to generate random numbers and arrays. These functions are useful for simulations, testing, and creating sample datasets.

Example: np.random.randint(1, 100, size=5) generates five random integers between 1 and 99.

Code and Output:

```
import numpy as np
```

```
arr = np.array([1,2,3])
```

```
print(arr)
```

Output:

```
[1 2 3]
```

```
arr1 = np.array([[1,2,3],[4,5,6]])
```

```
print(arr1)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

```
print("Shape of an array: ")
print(arr.shape)
print("Shape of an array1: ")
print(arr1.shape)
```

Output:

```
Shape of an array:
(3,)
Shape of an array1:
(2, 3)
```

```
vector = np.array([1,2,3,4,5,6])
matrix = vector.reshape(2,3)
print("Vector:")
print(vector)
print("Matrix:")
print(matrix)
```

Output:

```
Vector:
[1 2 3 4 5 6]
Matrix:
[[1 2 3]
 [4 5 6]]
```

```
vector1 = matrix.reshape(6)
print("Matrix:")
print(matrix)
print("Vector1:")
print(vector1)
```

Output:

```
Matrix:
[[1 2 3]
```

```
 [4 5 6]]
Vector1:
[1 2 3 4 5 6]
```

```
arr = np.arange(10)

print(arr)
```

Output:

```
[0 1 2 3 4 5 6 7 8 9]
```

```
b = np.arange(1,7)
print(b)
```

Output:

```
[1 2 3 4 5 6]
```

```
matrix2 = b.reshape(3,2)
print(matrix2)
```

Output:

```
[[1 2]
 [3 4]
 [5 6]]
```

```
zeros_array = np.zeros(5)
ones_array = np.ones(5)
print(zeros_array)
print(ones_array)
```

Output:

```
[0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1.]
```

```
zeros_array1 = np.zeros((2,3))
ones_array1 = np.ones((3,2))
print(zeros_array1)
print(ones_array1)
```

Output:

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```

```
arr1 = np.array([0,1,0,1])
print ("1st array : \n", arr1)
arr2 = np.array([1,0,1,0])
print ("2nd array : \n", arr2)
out_arr = np.column_stack((arr1,arr2))
```

```
print("Using column_stack: \n",out_arr)
```

Output:

```
1st array :
 [0 1 0 1]
2nd array :
 [1 0 1 0]
Using column_stack:
 [[0 1]
 [1 0]
 [0 1]
 [1 0]]
```

```
arr1 = np.array([0,1,0,1])
print ("1st array : \n", arr1)
arr2 = np.array([1,0,1,0])
print ("2nd array : \n", arr2)
out_arr = np.row_stack((arr1,arr2))
print("Using row_stack: \n",out_arr)
```

Output:

```
1st array :
 [0 1 0 1]
2nd array :
 [1 0 1 0]
Using row_stack:
 [[0 1 0 1]
 [1 0 1 0]]
```

```
a = np.arange(12).reshape((3,4))
print("3X4 Matrix: \n",a)
```

Output:

```
3X4 Matrix:
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
b = a + 150
print("Matrix after adding 150: \n",b)
```

Output:

```
Matrix after adding 150:
 [[150 151 152 153]
 [154 155 156 157]
 [158 159 160 161]]
```

```
c = a**2
print("Matrix after squaring each element: \n",c)
```

Output:

```
Matrix after squaring each element:
 [[  0   1   4   9]
 [ 16  25  36  49]
```

```
 [ 64  81 100 121]]
```

```
a = np.arange(1,21).reshape((4,5))*2
print("Required Array: \n",a)
```

Output:

```
Required  Array:
 [[ 2  4  6  8 10]
 [12 14 16 18 20]
 [22 24 26 28 30]
 [32 34 36 38 40]]
```

```
print("a > 6:")
print(a>6)
print("\n a == 10:")
print(a==10)
```

Output:

```
a > 6:
[[False False False  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]]

 a == 10:
[[False False False False  True]
 [False False False False False]
 [False False False False False]
 [False False False False False]]
```

```
print("(a < 3) | (a > 8):")
print((a < 3) | (a > 8))
print("\n(a > 4) & (a < 7):")
print((a>4) & (a<7))
print("\n ~(a > 6):")
print(~(a>6))
```

Output:

```
(a < 3) | (a > 8):
[[ True False False False  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]
 [ True  True  True  True  True]]

(a > 4) & (a < 7):
[[False False  True False False]
 [False False False False False]
 [False False False False False]
 [False False False False False]]

 ~(a > 6):
[[ True  True  True False False]
 [False False False False False]
 [False False False False False]
 [False False False False False]]
```

```
arr = np.arange(12)
print('Original array:')
print(arr)
```

Output:

```
Original array:
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

```
arr[7:12] = -1
print(arr)
```

Output:

```
[ 0  1  2  3  4  5  6 -1 -1 -1 -1 -1]
```

```
print(arr[[2,3,6]])
```

Output:

```
[2 3 6]
```

```
matrix = np.arange(1,13).reshape((3,4))
print(matrix)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
print(matrix[1])
```

Output:

```
[5 6 7 8]
```

```
print(matrix[:,2])
```

Output:

```
[ 3  7 11]
```

```
print(matrix[0:2])
```

Output:

```
[[1 2 3 4]
 [5 6 7 8]]
```

```
print(matrix[1,2])
```

Output:

```
7
```

```
print(matrix[0:2,0:3])
```

Output:

```
[[1 2 3]
 [5 6 7]]
```

```
matrix[2] = [1,2,3,4]
print(matrix)
```

Output:

```
[[1 2 3 4]
 [5 6 7 8]
 [1 2 3 4]]
```

```
names = np.array(["Rahul", "Simran", "Ritesh", "Simran", "Rahul"])
score = np.array([126,115,130,141,132])
print("names:",names)
print("score:",score)
```

Output:

```
names: ['Rahul' 'Simran' 'Ritesh' 'Simran' 'Rahul']
score: [126 115 130 141 132]
```

```
print("score < 130: ")
print(score[score < 130])
```

Output:

```
score < 130:
[126 115]
```

```
simran_score = score[names == "Simran"]
print("Simran's scores:",simran_score)
```

Output:

```
Simran's scores: [115 141]
```

```
score[names == "Rahul"] += 10
print("Updated Scores after adding 10 points to Rahul:",score)
```

Output:

```
Updated Scores after adding 10 points to Rahul: [136 115 130 141 142]
```

```
print("random.choice:")
print(np.random.choice([10,20,30,40,50]))
```

Output:

```
random.choice:
```

Output:

```
10
```

```
x = np.random.normal()
print(x)
```

Output:

```
0.2983623992872263
```

```
m = np.random.normal(loc=50, scale=5, size=5)
print(m)
```

Output:

```
[54.18732472 61.32449843 44.78469024 50.25140551 49.63730406]
```

```
x = np.random.binomial(n=10, p=0.5)
print(x)
```

Output:

```
5
```

```
num = np.random.uniform()
print(num)
```

Output:

```
0.7644028951459187
```

```
arr = np.random.uniform(size=5)
print(arr)
```

Output:

```
[0.57908597 0.66580234 0.21670364 0.62091917 0.74704922]
```

```
vals = np.random.uniform(10,20,size=5)
print(vals)
```

Output:

```
[10.66430641 11.191016   16.24869024 19.88247548 12.11299131]
```

```
np.random.seed(42)
print(np.random.randint(1,100,size=5))
```

Output:

```
[52 93 15 72 61]
```

```
matrix = np.arange(1,13).reshape((3,4))
```

```
print(matrix)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
print("sum of all elements: ")
print(np.sum(matrix))
```

Output:

```
sum of all elements:
```

Output:

```
78
```

```
print("sum of all rows(preserve columns): ")
print(np.sum(matrix, axis=0))
```

Output:

```
sum of all rows(preserve columns):
[15 18 21 24]
```

```
print("sum of all columns(preserve rows): ")
print(np.sum(matrix, axis=1))
```

Output:

```
sum of all columns(preserve rows):
[10 26 42]
```

```
# Additional statistical functions
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))
print("Standard Deviation:", np.std(a))
```

Output:

```
Mean:
```

Output:

```
 21.0
Median: 21.0
Maximum: 40
Minimum: 2
Standard Deviation: 11.532562594670797
```

Practical No.2

Exploratory Data Analysis using Pandas and Matplotlib

Aim:

To perform Exploratory Data Analysis (EDA) on the Telecom Churn dataset using Pandas and Matplotlib.

Theory

Exploratory Data Analysis (EDA) is the process of analyzing and understanding a dataset using statistical methods and visualizations before applying machine learning algorithms. It helps identify patterns, trends, missing values, outliers, and relationships between variables.

In this practical, Pandas and Matplotlib are used to perform EDA on the Telecom Churn dataset, which contains customer information such as tenure, monthly charges, total charges, contract type, and churn status.

1. Pandas

Pandas is a Python library used for data manipulation and analysis. It provides DataFrames to store and organize data in tabular form.

Common Pandas functions used in EDA:

- head() – Displays the first five rows.
- info() – Shows column names, data types, and non-null values.
- describe() – Provides statistical summaries of numerical columns.
- isnull().sum() – Checks missing values in each column.
- value_counts() – Counts the occurrences of unique values.
- groupby() – Groups data for analysis.

2. Matplotlib

Matplotlib is a Python library used to create graphs and charts for visualizing data. It helps represent complex data in an understandable graphical format.

Common visualizations include:

- Bar chart: Compares customer counts across categories.
- Histogram: Shows the distribution of numerical data.
- Pie chart: Displays proportions of different categories.
- Box plot: Identifies data distribution and potential outliers.
- Line chart: Shows trends in data.

3. Telecom Churn Dataset

Customer churn refers to customers discontinuing a company's services. The Telecom Churn dataset is used to understand customer behavior and identify factors associated with customers leaving a telecom service.

Important attributes may include:

- Customer ID: Unique identifier of each customer.
- Tenure: Duration for which a customer has used the service.
- Monthly Charges: Amount charged to a customer each month.
- Total Charges: Total amount charged over the customer's service period.
- Contract: Type of customer contract.
- Churn: Indicates whether a customer has left the service.

4. Steps in Exploratory Data Analysis

1. Import Pandas and Matplotlib libraries.
2. Load the Telecom Churn dataset into a DataFrame.
3. Display the first few rows and inspect the dataset structure.
4. Check the number of rows and columns and identify data types.
5. Find missing values and duplicate records.
6. Calculate descriptive statistics.
7. Analyze customer churn using frequency counts and grouping.
8. Create graphs to visualize customer characteristics and churn patterns.
9. Interpret the results to identify useful patterns and trends.

5. Importance of EDA

- Helps understand the structure and quality of data.
- Detects missing values, duplicates, and outliers.
- Identifies patterns and relationships between variables.
- Supports better feature selection for machine learning.
- Helps make data-driven business decisions, such as identifying customer groups with higher churn.

Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('/content/Lab-2-Data-Telecom_Churn (1).csv')
df.head()
```

Output:

```
  state  account length  area code phone number international plan  \
0    KS             128        415     382-4657                 no
1    OH             107        415     371-7191                 no
2    NJ             137        415     358-1921                 no
3    OH              84        408     375-9999                yes
4    OK              75        415     330-6626                yes

  voice mail plan  number vmail messages  total day minutes  total day calls  \
0             yes                     25              265.1              110
1             yes                     26              161.6              123
2              no                      0              243.4              114
3              no                      0              299.4               71
4              no                      0              166.7              113

   total day charge  ...  total eve calls  total eve charge  \
0             45.07  ...               99             16.78
1             27.47  ...              103             16.62
2             41.38  ...              110             10.30
3             50.90  ...               88              5.26
4             28.34  ...              122             12.61

   total night minutes  total night calls  total night charge  \
0                244.7                 91               11.01
1                254.4                103               11.45
2                162.6                104                7.32
3                196.9                 89                8.86
4                186.9                121                8.41

   total intl minutes  total intl calls  total intl charge  \
0                10.0                 3               2.70
1                13.7                 3               3.70
2                12.2                 5               3.29
3                 6.6                 7               1.78
4                10.1                 3               2.73

   customer service calls  churn
0                       1  False
1                       1  False
2                       0  False
3                       2  False
4                       3  False

[5 rows x 21 columns]
```

```
df.shape
```

Output:

```
(3333, 21)
```

```
df.columns
```

Output:

```
Index(['state', 'account length', 'area code', 'phone number',
       'international plan', 'voice mail plan', 'number vmail messages',
       'total day minutes', 'total day calls', 'total day charge',
       'total eve minutes', 'total eve calls', 'total eve charge',
       'total night minutes', 'total night calls', 'total night charge',
       'total intl minutes', 'total intl calls', 'total intl charge',
       'customer service calls', 'churn'],
      dtype='object')
```

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3333 entries, 0 to 3332
Data columns (total 21 columns):
 #   Column                  Non-Null Count  Dtype  
    ------                                  -----  
 0   state                   3333 non-null   object 
 1   account length          3333 non-null   int64  
 2   area code               3333 non-null   int64  
 3   phone number            3333 non-null   object 
 4   international plan      3333 non-null   object 
 5   voice mail plan         3333 non-null   object 
 6   number vmail messages   3333 non-null   int64  
 7   total day minutes       3333 non-null   float64
 8   total day calls         3333 non-null   int64  
 9   total day charge        3333 non-null   float64
 10  total eve minutes       3333 non-null   float64
 11  total eve calls         3333 non-null   int64  
 12  total eve charge        3333 non-null   float64
 13  total night minutes     3333 non-null   float64
 14  total night calls       3333 non-null   int64  
 15  total night charge      3333 non-null   float64
 16  total intl minutes      3333 non-null   float64
 17  total intl calls        3333 non-null   int64  
 18  total intl charge       3333 non-null   float64
 19  customer service calls  3333 non-null   int64  
 20  churn                   3333 non-null   bool   
dtypes: bool(1), float64(8), int64(8), object(4)
memory usage: 524.2+ KB
```

```
df.describe()
```

Output:

```
       account length    area code  number vmail messages  total day minutes  \
count     3333.000000  3333.000000            3333.000000        3333.000000   
mean       101.064806   437.182418               8.099010         179.775098   
std         39.822106    42.371290              13.688365          54.467389   
min          1.000000   408.000000               0.000000           0.000000   
25%         74.000000   408.000000               0.000000         143.700000   
50%        101.000000   415.000000               0.000000         179.400000   
75%        127.000000   510.000000              20.000000         216.400000   
max        243.000000   510.000000              51.000000         350.800000   

       total day calls  total day charge  total eve minutes  total eve calls  \
```

```
count      3333.000000       3333.000000        3333.000000      3333.000000
mean        100.435644         30.562307         200.980348       100.114311
std          20.069084          9.259435          50.713844        19.922625
min           0.000000          0.000000           0.000000         0.000000
25%          87.000000         24.430000         166.600000        87.000000
50%         101.000000         30.500000         201.400000       100.000000
75%         114.000000         36.790000         235.300000       114.000000
max         165.000000         59.640000         363.700000       170.000000

       total eve charge  total night minutes  total night calls  \
count       3333.000000          3333.000000        3333.000000
mean          17.083540           200.872037         100.107711
std            4.310668            50.573847          19.568609
min            0.000000            23.200000          33.000000
25%           14.160000           167.000000          87.000000
50%           17.120000           201.200000         100.000000
75%           20.000000           235.300000         113.000000
max           30.910000           395.000000         175.000000

       total night charge  total intl minutes  total intl calls  \
count         3333.000000         3333.000000       3333.000000
mean             9.039325           10.237294          4.479448
std              2.275873            2.791840          2.461214
min              1.040000            0.000000          0.000000
25%              7.520000            8.500000          3.000000
50%              9.050000           10.300000          4.000000
75%             10.590000           12.100000          6.000000
max             17.770000           20.000000         20.000000

       total intl charge  customer service calls
count        3333.000000             3333.000000
mean            2.764581                1.562856
std             0.753773                1.315491
min             0.000000                0.000000
25%             2.300000                1.000000
50%             2.780000                1.000000
75%             3.270000                2.000000
max             5.400000                9.000000
```

```
df['churn'] = df['churn'].astype(int)
df.head()
```

Output:

```
  state  account length  area code phone number international plan  \
0    KS             128        415     382-4657                 no
1    OH             107        415     371-7191                 no
2    NJ             137        415     358-1921                 no
3    OH              84        408     375-9999                yes
4    OK              75        415     330-6626                yes

  voice mail plan  number vmail messages  total day minutes  total day calls  \
0             yes                     25              265.1              110
1             yes                     26              161.6              123
2              no                      0              243.4              114
3              no                      0              299.4               71
4              no                      0              166.7              113

   total day charge  ...  total eve calls  total eve charge  \
0             45.07  ...               99             16.78
1             27.47  ...              103             16.62
```

```
2             41.38  ...               110             10.30
3             50.90  ...                88              5.26
4             28.34  ...               122             12.61

   total night minutes  total night calls  total night charge  \
0                244.7                 91               11.01
1                254.4                103               11.45
2                162.6                104                7.32
3                196.9                 89                8.86
4                186.9                121                8.41

   total intl minutes  total intl calls  total intl charge  \
0                10.0                 3               2.70
1                13.7                 3               3.70
2                12.2                 5               3.29
3                 6.6                 7               1.78
4                10.1                 3               2.73

   customer service calls  churn
0                       1      0
1                       1      0
2                       0      0
3                       2      0
4                       3      0

[5 rows x 21 columns]
```

```
print("\nStatistics of Categorical Features:")
display(df.describe(include='object'))
```

Output:

```
Statistics of Categorical Features:
```

Output:

```
       state phone number international plan voice mail plan
count   3333         3333                3333            3333
unique    51         3333                   2               2
top       WV     400-4344                  no              no
freq     106            1                3010            2411
```

```
# Statistics of all features
print("\nStatistics of All Features:")
display(df.describe(include='all'))
```

Output:

```
Statistics of All Features:
```

Output:

```
       state  account length    area code phone number international plan  \
count   3333     3333.000000  3333.000000         3333                3333
unique    51             NaN          NaN         3333                   2
top       WV             NaN          NaN     400-4344                  no
freq     106             NaN          NaN            1                3010
mean     NaN      101.064806   437.182418          NaN                 NaN
```

```
std       NaN       39.822106    42.371290          NaN                NaN
min       NaN        1.000000   408.000000          NaN                NaN
25%       NaN       74.000000   408.000000          NaN                NaN
50%       NaN      101.000000   415.000000          NaN                NaN
75%       NaN      127.000000   510.000000          NaN                NaN
max       NaN      243.000000   510.000000          NaN                NaN

       voice mail plan  number vmail messages  total day minutes  \
count             3333            3333.000000        3333.000000
unique               2                    NaN                NaN
top                 no                    NaN                NaN
freq              2411                    NaN                NaN
mean               NaN               8.099010         179.775098
std                NaN              13.688365          54.467389
min                NaN               0.000000           0.000000
25%                NaN               0.000000         143.700000
50%                NaN               0.000000         179.400000
75%                NaN              20.000000         216.400000
max                NaN              51.000000         350.800000

        total day calls  total day charge  ...  total eve calls  \
count       3333.000000       3333.000000  ...       3333.000000
unique              NaN               NaN  ...              NaN
top                 NaN               NaN  ...              NaN
freq                NaN               NaN  ...              NaN
mean         100.435644         30.562307  ...       100.114311
std           20.069084          9.259435  ...        19.922625
min            0.000000          0.000000  ...         0.000000
25%           87.000000         24.430000  ...        87.000000
50%          101.000000         30.500000  ...       100.000000
75%          114.000000         36.790000  ...       114.000000
max          165.000000         59.640000  ...       170.000000

        total eve charge  total night minutes  total night calls  \
count        3333.000000          3333.000000        3333.000000
unique               NaN                  NaN                NaN
top                  NaN                  NaN                NaN
freq                 NaN                  NaN                NaN
mean           17.083540           200.872037         100.107711
std             4.310668            50.573847          19.568609
min             0.000000            23.200000          33.000000
25%            14.160000           167.000000          87.000000
50%            17.120000           201.200000         100.000000
75%            20.000000           235.300000         113.000000
max            30.910000           395.000000         175.000000

        total night charge  total intl minutes  total intl calls  \
count          3333.000000         3333.000000       3333.000000
unique                 NaN                 NaN               NaN
top                    NaN                 NaN               NaN
freq                   NaN                 NaN               NaN
mean              9.039325           10.237294          4.479448
std               2.275873            2.791840          2.461214
min               1.040000            0.000000          0.000000
25%               7.520000            8.500000          3.000000
50%               9.050000           10.300000          4.000000
75%              10.590000           12.100000          6.000000
max              17.770000           20.000000         20.000000

        total intl charge  customer service calls        churn
```

```
count         3333.000000              3333.000000  3333.000000
unique                NaN                       NaN          NaN
top                   NaN                       NaN          NaN
freq                  NaN                       NaN          NaN
mean             2.764581                  1.562856     0.144914
std              0.753773                  1.315491     0.352067
min              0.000000                  0.000000     0.000000
25%              2.300000                  1.000000     0.000000
50%              2.780000                  1.000000     0.000000
75%              3.270000                  2.000000     0.000000
max              5.400000                  9.000000     1.000000

[11 rows x 21 columns]
```

```
df['churn'].value_counts()
```

Output:

```
churn
0    2850
1     483
Name: count, dtype: int64
```

```
df['total day charge'].sort_values(ascending=False)
```

Output:

```
365     59.64
985     58.96
2594    58.70
156     57.36
605     57.04
        ...
1986     1.34
2753     1.33
2736     0.44
1397     0.00
1345     0.00
Name: total day charge, Length: 3333, dtype: float64
```

```
churn_proportion = df['churn'].mean()
churn_proportion
```

Output:

```
np.float64(0.14491449144914492)
```

```
churned_user = df[df['churn'] == 1]
numerical_cols = churned_user.select_dtypes(include='number').columns

numerical_cols = numerical_cols.drop('churn')
churned_user[numerical_cols].mean()
```

Output:

```
account length            102.664596
area code                 437.817805
number vmail messages       5.115942
total day minutes         206.914079
total day calls           101.335404
total day charge           35.175921
total eve minutes         212.410145
```

```
total eve calls            100.561077
total eve charge            18.054969
total night minutes        205.231677
total night calls          100.399586
total night charge           9.235528
total intl minutes          10.700000
total intl calls             4.163561
total intl charge            2.889545
customer service calls       2.229814
dtype: float64
```

```
churned_user['total day minutes'].mean()
```

Output:

```
np.float64(206.91407867494823)
```

```
df[df['churn'] == 0 & (df['international plan'] == 'No')]['total intl minutes'].max()
```

Output:

```
18.9
```

```
# Reload the original dataset
df = pd.read_csv('/content/Lab-2-Data-Telecom_Churn (1).csv')
```

```
df
```

Output:

```
     state  account length   area code phone number international plan  \
0        KS              128         415     382-4657                 no
1        OH              107         415     371-7191                 no
2        NJ              137         415     358-1921                 no
3        OH               84         408     375-9999                yes
4        OK               75         415     330-6626                yes
...     ...              ...         ...          ...                ...
3328     AZ              192         415     414-4276                 no
3329     WV               68         415     370-3271                 no
3330     RI               28         510     328-8230                 no
3331     CT              184         510     364-6381                yes
3332     TN               74         415     400-4344                 no

     voice mail plan  number vmail messages  total day minutes  \
0                yes                      25              265.1
1                yes                      26              161.6
2                 no                       0              243.4
3                 no                       0              299.4
4                 no                       0              166.7
...              ...                     ...                ...
3328             yes                      36              156.2
3329              no                       0              231.1
3330              no                       0              180.8
3331              no                       0              213.8
3332             yes                      25              234.4

      total day calls  total day charge  ...  total eve calls  \
0                 110             45.07  ...               99
1                 123             27.47  ...              103
2                 114             41.38  ...              110
3                  71             50.90  ...               88
```

```
4                 113             28.34  ...               122
...               ...               ...  ...               ...
3328               77             26.55  ...               126
3329               57             39.29  ...                55
3330              109             30.74  ...                58
3331              105             36.35  ...                84
3332              113             39.85  ...                82

      total eve charge  total night minutes  total night calls  \
0                16.78                244.7                 91
1                16.62                254.4                103
2                10.30                162.6                104
3                 5.26                196.9                 89
4                12.61                186.9                121
...                ...                  ...                ...
3328             18.32                279.1                 83
3329             13.04                191.3                123
3330             24.55                191.9                 91
3331             13.57                139.2                137
3332             22.60                241.4                 77

      total night charge  total intl minutes  total intl calls  \
0                  11.01                10.0                 3
1                  11.45                13.7                 3
2                   7.32                12.2                 5
3                   8.86                 6.6                 7
4                   8.41                10.1                 3
...                  ...                 ...               ...
3328               12.56                 9.9                 6
3329                8.61                 9.6                 4
3330                8.64                14.1                 6
3331                6.26                 5.0                10
3332               10.86                13.7                 4

      total intl  charge  customer service calls  churn
0                   2.70                       1  False
1                   3.70                       1  False
2                   3.29                       0  False
3                   1.78                       2  False
4                   2.73                       3  False
...                  ...                     ...    ...
3328                2.67                       2  False
3329                2.59                       3  False
3330                3.81                       2  False
3331                1.35                       2  False
3332                3.70                       0  False

[3333 rows x 21 columns]
```

```
df['international plan'].unique()
```

Output:

```
array(['no', 'yes'], dtype=object)
```

```
df['international plan'] = (
    df['international plan']
    .astype(str)
    .str.strip()
    .str.lower()
```

```
    .map({'yes': True, 'no': False})
)
df.head()
```

Output:

```
  state  account length  area code phone number  international plan  \
0    KS             128        415     382-4657                False
1    OH             107        415     371-7191                False
2    NJ             137        415     358-1921                False
3    OH              84        408     375-9999                 True
4    OK              75        415     330-6626                 True

  voice mail plan  number vmail messages  total day minutes  total day calls  \
0             yes                     25              265.1              110
1             yes                     26              161.6              123
2              no                      0              243.4              114
3              no                      0              299.4               71
4              no                      0              166.7              113

   total day charge  ...  total eve calls  total eve charge  \
0             45.07  ...               99             16.78
1             27.47  ...              103             16.62
2             41.38  ...              110             10.30
3             50.90  ...               88              5.26
4             28.34  ...              122             12.61

   total night minutes  total night calls  total night charge  \
0                244.7                 91               11.01
1                254.4                103               11.45
2                162.6                104                7.32
3                196.9                 89                8.86
4                186.9                121                8.41

   total intl minutes  total intl calls  total intl charge  \
0                10.0                 3               2.70
1                13.7                 3               3.70
2                12.2                 5               3.29
3                 6.6                 7               1.78
4                10.1                 3               2.73

   customer service calls  churn
0                       1      0
1                       1      0
2                       0      0
3                       2      0
4                       3      0

[5 rows x 21 columns]
```

```
df['voice mail plan'] = (
    df['voice mail plan']
    .astype(str)
    .str.strip()
    .str.lower()
    .map({'yes': True, 'no': False})
)
df.head()
```

Output:

```
  state  account length  area code phone number  international plan  \
```

```
0    KS              128         415      382-4657                False
1    OH              107         415      371-7191                False
2    NJ              137         415      358-1921                False
3    OH               84         408      375-9999                 True
4    OK               75         415      330-6626                 True

   voice mail plan  number vmail messages  total day minutes  total day calls  \
0             True                      25              265.1              110
1             True                      26              161.6              123
2            False                       0              243.4              114
3            False                       0              299.4               71
4            False                       0              166.7              113

   total day charge  ...  total eve calls  total eve charge  \
0             45.07  ...               99             16.78
1             27.47  ...              103             16.62
2             41.38  ...              110             10.30
3             50.90  ...               88              5.26
4             28.34  ...              122             12.61

   total night minutes  total night calls   total night charge  \
0                244.7                 91                11.01
1                254.4                103                11.45
2                162.6                104                 7.32
3                196.9                 89                 8.86
4                186.9                121                 8.41

   total intl minutes  total intl calls  total intl charge  \
0                10.0                 3               2.70
1                13.7                 3               3.70
2                12.2                 5               3.29
3                 6.6                 7               1.78
4                10.1                 3               2.73

   customer service calls  churn
0                       1      0
1                       1      0
2                       0      0
3                       2      0
4                       3      0

[5 rows x 21 columns]
```

```
gb = df.groupby('churn')['total day minutes'].describe()
gb
```

Output:

```
        count        mean         std  min       25%    50%     75%    max
churn
False  2850.0  175.175754  50.181655  0.0  142.825  177.2  210.30  315.6
True    483.0  206.914079  68.997792  0.0  153.250  217.6  265.95  350.8
```

```
gb = df.groupby('churn')['total eve minutes'].describe()
gb
```

Output:

```
        count        mean         std    min    25%    50%     75%    max
churn
False  2850.0  199.043298  50.292175    0.0  164.5  199.6  233.20  361.8
```

```
True    483.0  212.410145  51.728910  70.9  177.1  211.3  249.45  363.7
```

```
gb = df.groupby('churn')['total night minutes'].describe()
gb
```

Output:

```
        count        mean         std   min     25%     50%     75%    max
churn
False  2850.0  200.133193  51.105032  23.2  165.90  200.25  234.90  395.0
True    483.0  205.231677  47.132825  47.4  171.25  204.80  239.85  354.9
```

```
crosstab = pd.crosstab(df['churn'],df['international plan'],normalize=True)
crosstab
```

Output:

```
international plan     False     True
churn
False                0.79928  0.055806
True                 0.10381  0.041104
```

```
data = df.groupby('area code')
pivot = data['total day calls'].mean().reset_index()
pivot
```

Output:

```
   area code  total day calls
0        408       100.496420
1        415       100.576435
2        510       100.097619
```

```
data = df.groupby('area code')
pivot = data['total eve calls'].mean().reset_index()
pivot
```

Output:

```
   area code  total eve calls
0        408        99.788783
1        415       100.503927
2        510        99.671429
```

```
data = df.groupby('area code')
pivot = data['total night calls'].mean().reset_index()
pivot
```

Output:

```
   area code  total night calls
0        408          99.039379
1        415         100.398187
2        510         100.601190
```

```
df['total_calls'] = df['total day calls'] + df['total eve calls'] + df['total night calls'] + df['total intl
calls']
print(df['total_calls'])
df.head()
```

Output:

```
0       303
```

```
1       332
2       333
3       255
4       359
       ...
3328    292
3329    239
3330    264
3331    336
3332    276
Name: total_calls, Length: 3333, dtype: int64
```

Output:

```
  state  account length  area code phone number  international plan  \
0    KS             128        415     382-4657                False
1    OH             107        415     371-7191                False
2    NJ             137        415     358-1921                False
3    OH              84        408     375-9999                 True
4    OK              75        415     330-6626                 True

   voice mail plan  number vmail messages  total day minutes  total day calls  \
0             True                     25              265.1              110
1             True                     26              161.6              123
2            False                      0              243.4              114
3            False                      0              299.4               71
4            False                      0              166.7              113

   total day charge  ...  total eve charge  total night minutes  \
0             45.07  ...             16.78                244.7
1             27.47  ...             16.62                254.4
2             41.38  ...             10.30                162.6
3             50.90  ...              5.26                196.9
4             28.34  ...             12.61                186.9

   total night calls  total night charge  total intl minutes  \
0                 91               11.01                10.0
1                103               11.45                13.7
2                104                7.32                12.2
3                 89                8.86                 6.6
4                121                8.41                10.1

   total intl calls  total intl charge  customer service calls  churn  \
0                 3               2.70                       1      0
1                 3               3.70                       1      0
2                 5               3.29                       0      0
3                 7               1.78                       2      0
4                 3               2.73                       3      0

   total_calls
0          303
1          332
2          333
3          255
4          359

[5 rows x 22 columns]
```

```
df['total_charge'] = (df['total day charge'] + df['total eve charge'] + df['total intl charge'] + df['total
night charge'])
```

```
print(df['total_charge'])
df
```

Output:

```
0       75.56
1       59.24
2       62.29
3       66.80
4       52.09
        ...
3328    60.10
3329    63.53
3330    67.74
3331    57.53
3332    77.01
Name: total_charge, Length: 3333, dtype: float64
```

Output:

```
     state  account length   area code phone number  international plan  \
0       KS             128         415     382-4657                False
1       OH             107         415     371-7191                False
2       NJ             137         415     358-1921                False
3       OH              84         408     375-9999                 True
4       OK              75         415     330-6626                 True
...    ...             ...         ...          ...                  ...
3328    AZ             192         415     414-4276                False
3329    WV              68         415     370-3271                False
3330    RI              28         510     328-8230                False
3331    CT             184         510     364-6381                 True
3332    TN              74         415     400-4344                False

      voice mail plan  number vmail messages  total day minutes  \
0                True                      25              265.1
1                True                      26              161.6
2               False                       0              243.4
3               False                       0              299.4
4               False                       0              166.7
...               ...                     ...                ...
3328             True                      36              156.2
3329            False                       0              231.1
3330            False                       0              180.8
3331            False                       0              213.8
3332             True                      25              234.4

      total day calls  total day charge  ...  total night  minutes  \
0                 110             45.07  ...                244.7
1                 123             27.47  ...                254.4
2                 114             41.38  ...                162.6
3                  71             50.90  ...                196.9
4                 113             28.34  ...                186.9
...               ...               ...  ...                  ...
3328               77             26.55  ...                279.1
3329               57             39.29  ...                191.3
3330              109             30.74  ...                191.9
3331              105             36.35  ...                139.2
3332              113             39.85  ...                241.4

      total night calls  total night charge  total intl minutes  \
```

```
0                    91               11.01                10.0
1                   103               11.45                13.7
2                   104                7.32                12.2
3                    89                8.86                 6.6
4                   121                8.41                10.1
...                 ...                 ...                 ...
3328                 83               12.56                 9.9
3329                123                8.61                 9.6
3330                 91                8.64                14.1
3331                137                6.26                 5.0
3332                 77               10.86                13.7

      total intl calls  total intl charge   customer service calls  churn  \
0                    3               2.70                        1  False
1                    3               3.70                        1  False
2                    5               3.29                        0  False
3                    7               1.78                        2  False
4                    3               2.73                        3  False
...                ...                ...                      ...    ...
3328                 6               2.67                        2  False
3329                 4               2.59                        3  False
3330                 6               3.81                        2  False
3331                10               1.35                        2  False
3332                 4               3.70                        0  False

      total_calls  total_charge
0             303         75.56
1             332         59.24
2             333         62.29
3             255         66.80
4             359         52.09
...           ...           ...
3328          292         60.10
3329          239         63.53
3330          264         67.74
3331          336         57.53
3332          276         77.01

[3333 rows x 23 columns]
```

```
pd.crosstab(df['churn'],df['international  plan'])
```

Output:

```
international plan   False  True
churn
False                 2664    186
True                   346    137
```

```
pd.crosstab(df['customer service calls'],df['churn'])
```

Output:

```
churn                    False  True
customer service calls
0                          605     92
1                         1059    122
2                          672     87
3                          385     44
4                           90     76
5                           26     40
```

```
6                            8     14
7                            4      5
8                            1      1
9                            0      2
```

```
pd.crosstab(df['customer  service calls'] > 3,df['churn'])
```

Output:

```
churn                   False  True
customer service calls
False                    2721    345
True                      129    138
```

```
pd.crosstab(df['international plan'], df['customer service calls'] > 3, values=df['churn'], aggfunc=np.mean)
```

Output:

```
customer service calls     False     True
international plan
False                    0.08192  0.497908
True                     0.40000  0.678571
```

Practical No.3

Linear Regression using Gradient Descent

Aim:

To implement multiple linear regression using gradient descent from scratch on the USA Housing dataset, and to study the effect of different learning rates (0.1, 0.01, 0.001) on convergence.

Theory

Linear Regression is a supervised machine learning algorithm used to predict a continuous numerical value based on one or more input variables. In this practical, multiple linear regression is implemented from scratch using Gradient Descent on the USA Housing dataset to predict house prices based on different features.

1. Linear Regression

Linear regression establishes a mathematical relationship between input variables and a target variable.

In Multiple Linear Regression, two or more independent variables are used to predict a dependent variable.

For the USA Housing dataset, features may include average area income, house age, number of rooms, number of bedrooms, and population.

2. Gradient Descent

Gradient Descent is an optimization algorithm used to minimize the error between actual and predicted values. It updates the model parameters iteratively to find values that reduce the cost function.

The parameter update equation is:

3. Cost Function

The Mean Squared Error (MSE) measures the average squared difference between actual and predicted house prices.

4. Learning Rate

The learning rate determines how large each parameter update is during Gradient Descent. In this practical, three learning rates are compared.

Learning rate	Description
0.1	Large updates; may converge quickly or overshoot and diverge.
0.01	Moderate updates; may provide stable convergence depending on feature scaling.
0.001	Small updates; generally slower convergence but may provide stable learning.

The actual behavior depends on feature scaling and the dataset. Therefore, input features are commonly standardized before applying Gradient Descent.

Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

sns.set_style("darkgrid")
%matplotlib inline
```

```
import os
import urllib.request

CSV_PATH = "USA_Housing.csv"
URL = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-
Python/master/Datasets/USA_Housing.csv"

if not os.path.exists(CSV_PATH):
    try:
        urllib.request.urlretrieve(URL, CSV_PATH)
        print("Dataset downloaded successfully.")
    except Exception as e:
        print("Download failed:", e)
        print("Please manually download 'USA_Housing.csv' (e.g. from Kaggle: "
              "https://www.kaggle.com/datasets/vedavyasv/usa-housing) "
              "and place it in the same folder as this notebook.")
else:
    print("Using local copy of USA_Housing.csv")

df = pd.read_csv(CSV_PATH)
df.head()
```

Output:

```
Using local copy of USA_Housing.csv
```

Output:

```
   Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms  \
0      79545.458574             5.682861                   7.009188
1      79248.642455             6.002900                   6.730821
2      61287.067179             5.865890                   8.512727
3      63345.240046             7.188236                   5.586729
4      59982.197226             5.040555                   7.839388

   Avg. Area Number of Bedrooms  Area Population          Price  \
0                          4.09     23086.800503  1.059034e+06
1                          3.09     40173.072174  1.505891e+06
2                          5.13     36882.159400  1.058988e+06
3                          3.26     34310.242831  1.260617e+06
4                          4.23     26354.109472  6.309435e+05

                                          Address
```

```
0  208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1  188 Johnson Views Suite 079\nLake Kathleen, CA...
2  9127 Elizabeth Stravenue\nDanieltown, WI 06482...
3                          USS Barnett\nFPO AP 44820
4                         USNS Raymond\nFPO AE 09386
```

```
print("Shape of the dataset:", df.shape)
df.info()
```

Output:

```
Shape of the dataset: (5000, 7)
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                        Non-Null Count  Dtype
---
 0   Avg. Area Income              5000 non-null   float64
 1   Avg. Area House Age           5000 non-null   float64
 2   Avg. Area Number of Rooms     5000 non-null   float64
 3   Avg. Area Number of Bedrooms  5000 non-null   float64
 4   Area Population               5000 non-null   float64
 5   Price                         5000 non-null   float64
 6   Address                       5000 non-null   str
dtypes: float64(6), str(1)
memory usage: 273.6 KB
```

```
df.describe()
```

Output:

```
       Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms   \
count       5000.000000          5000.000000                5000.000000
mean       68583.108984             5.977222                   6.987792
std        10657.991214             0.991456                   1.005833
min        17796.631190             2.644304                   3.236194
25%        61480.562388             5.322283                   6.299250
50%        68804.286404             5.970429                   7.002902
75%        75783.338666             6.650808                   7.665871
max       107701.748378             9.519088                  10.759588

       Avg. Area Number of Bedrooms  Area Population         Price
count                   5000.000000      5000.000000  5.000000e+03
mean                       3.981330     36163.516039  1.232073e+06
std                        1.234137      9925.650114  3.531176e+05
min                        2.000000       172.610686  1.593866e+04
25%                        3.140000     29403.928702  9.975771e+05
50%                        4.050000     36199.406689  1.232669e+06
75%                        4.490000     42861.290769  1.471210e+06
max                        6.500000     69621.713378  2.469066e+06
```

```
# Check for missing values
df.isnull().sum()
```

Output:

```
Avg. Area Income                0
Avg. Area House Age             0
Avg. Area Number of Rooms       0
Avg. Area Number of Bedrooms    0
Area Population                 0
```

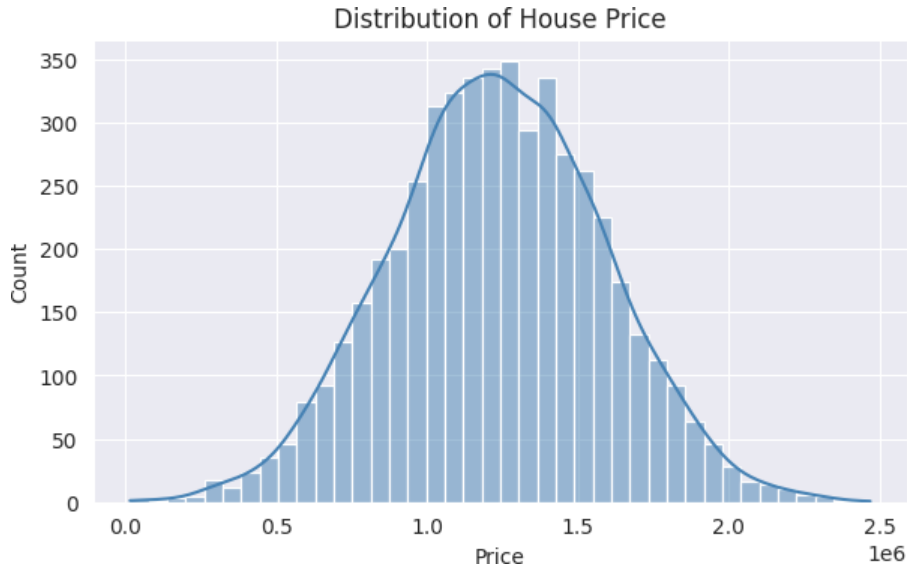
```
Price                          0
Address                        0
dtype: int64
```

```
# Check for duplicate rows
print("Duplicate rows:", df.duplicated().sum())
```

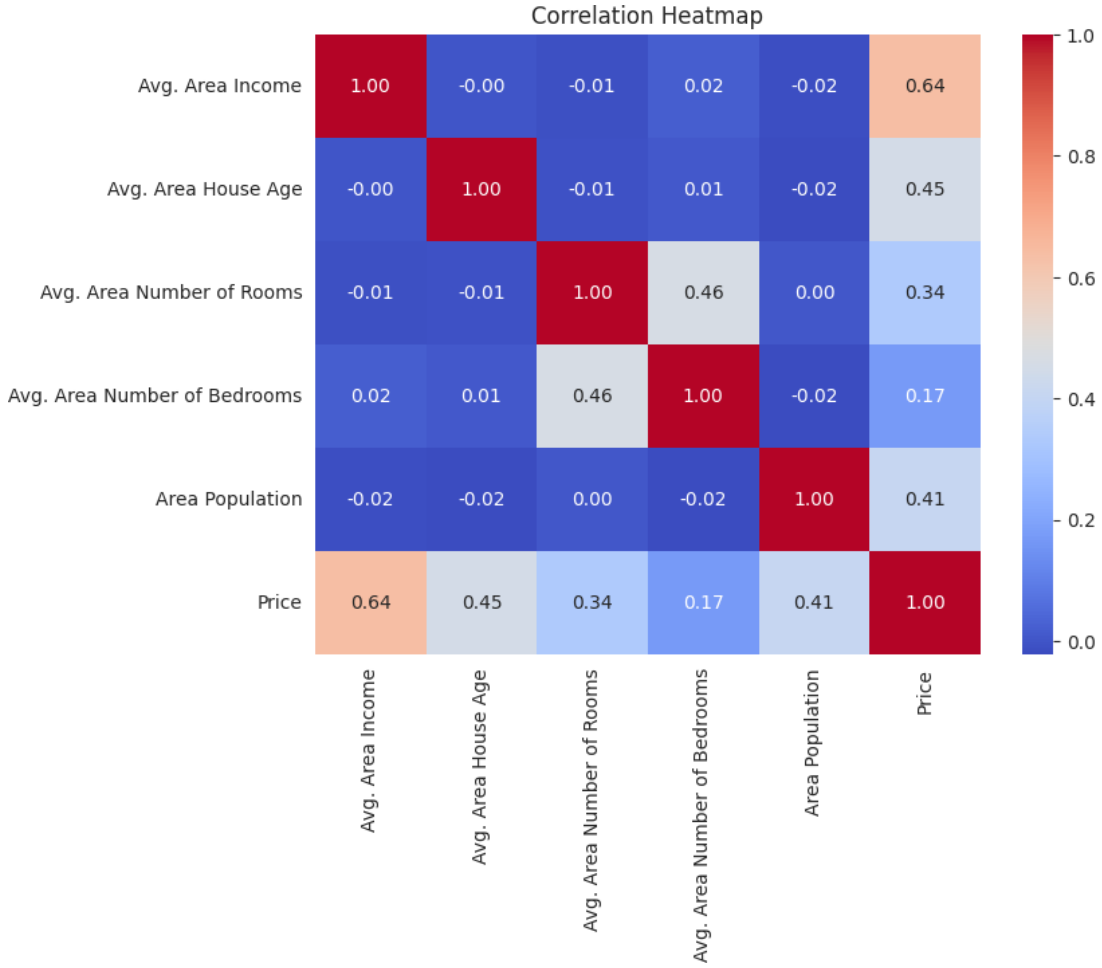
Output:

```
Duplicate rows: 0
```

```
# Distribution of the target variable
plt.figure(figsize=(7,4))
sns.histplot(df['Price'], kde=True, bins=40, color='steelblue')
plt.title("Distribution of House Price")
plt.xlabel("Price")
plt.show()
```




```
# Correlation heatmap (only numeric columns)
plt.figure(figsize=(8,6))
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```




```
# 1. Drop irrelevant column
data = df.drop(columns=['Address'])

# 2. Features and target
X = data.drop(columns=['Price'])
y = data['Price'].values.reshape(-1, 1)

print("Feature columns:", list(X.columns))
print("X shape:", X.shape, " y shape:", y.shape)
```

Output:

```
Feature columns: ['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms', 'Avg. Area Number of Bedrooms', 'Area Population']
X shape: (5000, 5)  y shape: (5000, 1)
```

```
# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])
```

Output:

```
Training samples: 4000
Testing samples : 1000
```

```
# 4. Feature Scaling (Standardization: mean = 0, std = 1)
```

```
scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

scaler_y = StandardScaler()
y_train_scaled = scaler_y.fit_transform(y_train)
y_test_scaled = scaler_y.transform(y_test)

print("Mean of scaled X_train (should be ~0):", np.round(X_train_scaled.mean(axis=0), 3))
print("Std of scaled X_train (should be ~1):", np.round(X_train_scaled.std(axis=0), 3))
```

Output:

```
Mean of scaled X_train (should be ~0): [-0. -0. -0. -0.  0.]
Std of scaled X_train (should be ~1): [1. 1. 1. 1. 1.]
```

```
def add_bias(X):
    '''Add a column of 1s to X for the bias/intercept term theta_0.'''
    return np.hstack([np.ones((X.shape[0], 1)), X])

def compute_cost(X, y, theta):
    '''Mean Squared Error cost function J(theta).'''
    m = len(y)
    predictions = X.dot(theta)
    errors = predictions - y
    cost = (1 / (2 * m)) * np.sum(errors ** 2)
    return cost

def gradient_descent(X, y, learning_rate=0.01, epochs=1000):
    '''
    Performs batch gradient descent.
    Returns the learned parameters (theta) and the history of cost per epoch.
    '''
    m, n = X.shape
    theta = np.zeros((n, 1))          # initialize weights to 0
    cost_history = []

    for epoch in range(epochs):
        predictions = X.dot(theta)
        errors = predictions - y
        gradients = (1 / m) * X.T.dot(errors)
        theta = theta - learning_rate * gradients

        cost = compute_cost(X, y, theta)
        cost_history.append(cost)

    return theta, cost_history

# Prepare data with bias term
X_train_b = add_bias(X_train_scaled)
X_test_b = add_bias(X_test_scaled)

print("X_train_b shape (with bias column):", X_train_b.shape)
```

Output:

```
X_train_b shape (with bias column): (4000, 6)
```

```
learning_rates = [0.1, 0.01, 0.001]
```

```
epochs = 500

results = {}   # stores theta and cost_history for each learning rate

for lr in learning_rates:
    theta, cost_history = gradient_descent(X_train_b, y_train_scaled, learning_rate=lr, epochs=epochs)
    results[lr] = {"theta": theta, "cost_history": cost_history}
    print(f"Learning Rate = {lr:<6} | Final Cost = {cost_history[-1]:.6f} | Initial Cost =
{cost_history[0]:.6f}")
```

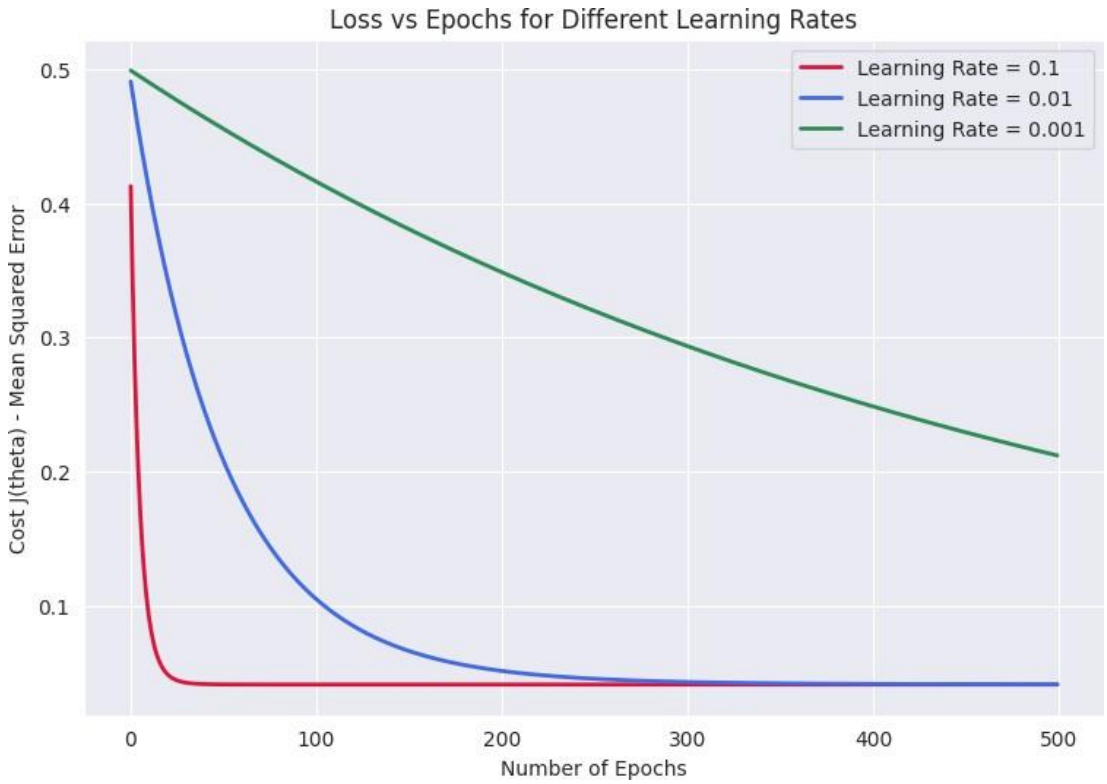
Output:

```
Learning Rate = 0.1    | Final Cost = 0.041011 | Initial Cost = 0.412762
Learning Rate = 0.01   | Final Cost = 0.041127 | Initial Cost = 0.490849
Learning Rate = 0.001  | Final Cost = 0.211883 | Initial Cost = 0.499081
```

```
plt.figure(figsize=(9,6))
colors = {0.1: 'crimson', 0.01: 'royalblue', 0.001: 'seagreen'}

for lr in learning_rates:
    plt.plot(results[lr]["cost_history"], label=f"Learning Rate = {lr}", color=colors[lr], linewidth=2)

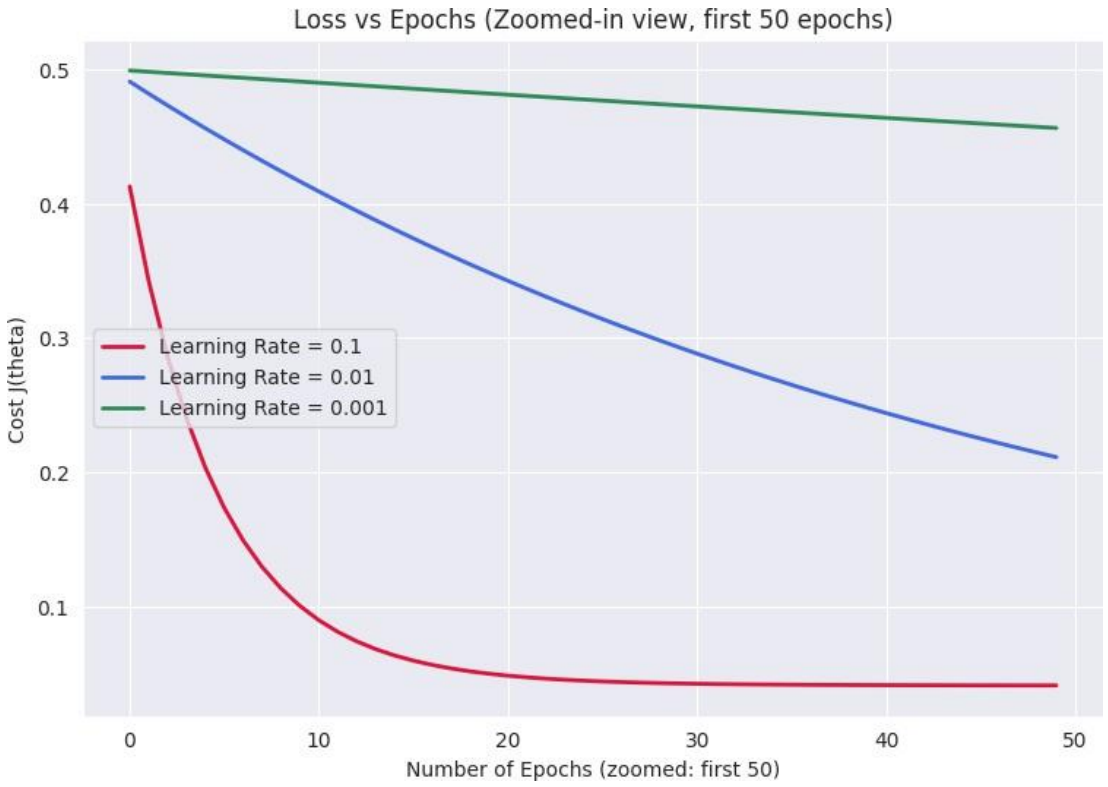
plt.xlabel("Number of Epochs")
plt.ylabel("Cost J(theta) - Mean Squared Error")
plt.title("Loss vs Epochs for Different Learning Rates")
plt.legend()
plt.show()
```




```
# Zoomed-in view of the first 50 epochs to see early convergence behaviour
plt.figure(figsize=(9,6))
for lr in learning_rates:
    plt.plot(results[lr]["cost_history"][:50], label=f"Learning Rate = {lr}", color=colors[lr], linewidth=2)

plt.xlabel("Number of Epochs (zoomed: first 50)")
```

```
plt.ylabel("Cost J(theta)")
plt.title("Loss vs Epochs (Zoomed-in view, first 50 epochs)")
plt.legend()
plt.show()
```




```
best_lr = 0.1
best_theta = results[best_lr]["theta"]

# Predictions on scaled test data
y_pred_scaled = X_test_b.dot(best_theta)

# Convert predictions back to original Price scale
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_true = scaler_y.inverse_transform(y_test_scaled)

rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2 = r2_score(y_true, y_pred)

print(f"Gradient Descent Model (learning rate = {best_lr})")
print(f"  RMSE : {rmse:,.2f}")
print(f"  R^2  : {r2:.4f}")
```

Output:

```
Gradient Descent Model (learning rate = 0.1)
  RMSE : 100,444.06
  R^2  : 0.9180
```

```
# Sanity check against sklearn's closed-form LinearRegression
sk_model = LinearRegression()
sk_model.fit(X_train_scaled, y_train_scaled.ravel())
y_pred_sklearn = scaler_y.inverse_transform(sk_model.predict(X_test_scaled).reshape(-1, 1))

rmse_sklearn = np.sqrt(mean_squared_error(y_true, y_pred_sklearn))
```

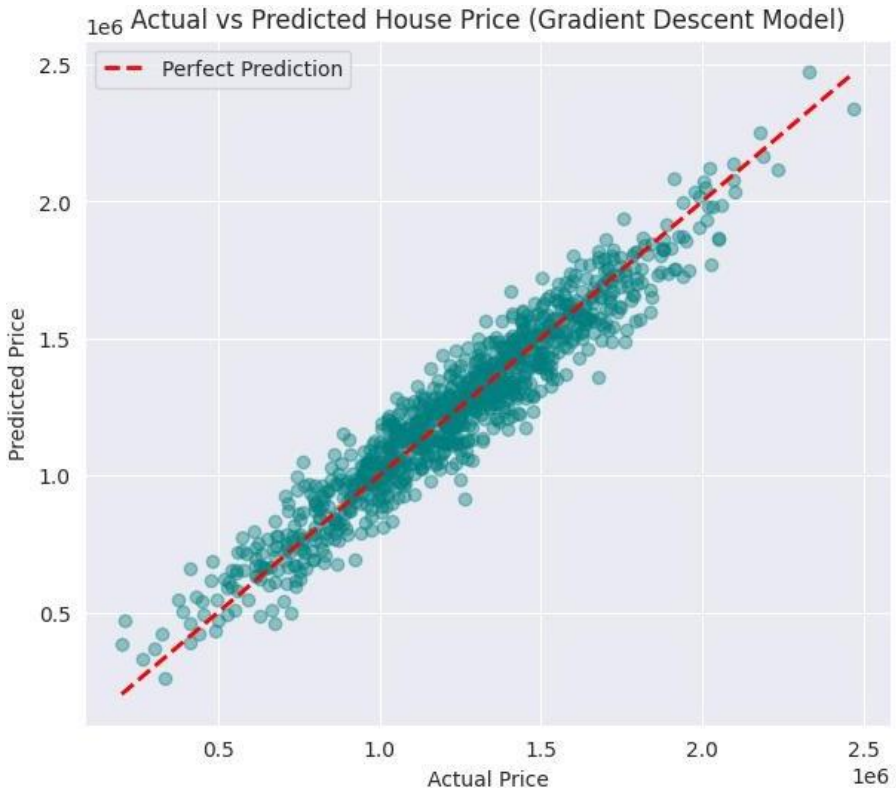
```
r2_sklearn = r2_score(y_true, y_pred_sklearn)

print("sklearn LinearRegression (for comparison)")
print(f"  RMSE : {rmse_sklearn:,.2f}")
print(f"  R^2  : {r2_sklearn:.4f}")
```

Output:

```
sklearn LinearRegression (for comparison)
  RMSE : 100,444.06
  R^2  : 0.9180
```

```
# Visualize: Actual vs Predicted Price
plt.figure(figsize=(7,6))
plt.scatter(y_true, y_pred, alpha=0.4, color='teal')
plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'r--', linewidth=2, label="Perfect
Prediction")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Price (Gradient Descent Model)")
plt.legend()
plt.show()
```



Practical No.4

Logistic Regression

Aim:

To implement Logistic Regression on the Breast Cancer dataset and evaluate it using the confusion matrix, accuracy, precision, recall, F1-score, different classification thresholds, and ROC/PR curves.

Theory

Logistic Regression is a supervised machine learning algorithm used for classification problems. It predicts the probability that a given input belongs to a particular class. In this practical, Logistic Regression is applied to the Breast Cancer dataset to classify tumors as malignant (cancerous) or benign (non-cancerous).

1. Logistic Regression

Unlike Linear Regression, which predicts continuous numerical values, Logistic Regression predicts probabilities between 0 and 1. These probabilities are converted into class labels using a classification threshold.

The predicted probability is between 0 and 1. For example, using a threshold of 0.5, a probability greater than or equal to 0.5 is assigned to the positive class, while a lower probability is assigned to the negative class.

2. Breast Cancer Dataset

The Breast Cancer dataset contains features extracted from cell nuclei, such as radius, texture, perimeter, area, smoothness, and compactness.

The target variable represents two classes:

- Malignant: Cancerous tumor.
- Benign: Non-cancerous tumor.

The dataset is used to train and evaluate a classification model. The exact positive-class label depends on the dataset encoding.

3. Confusion Matrix

A confusion matrix compares actual class labels with predicted class labels.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- TP: Positive cases correctly identified.
- TN: Negative cases correctly identified.
- FP: Negative cases incorrectly predicted as positive.
- FN: Positive cases incorrectly predicted as negative.

4. Evaluation Metrics

These metrics help evaluate different aspects of model performance. In medical classification, recall for malignant cases is especially important because false negatives may mean missing a cancerous tumor.

Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from  sklearn.datasets  import  load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, roc_curve, roc_auc_score,
                             precision_recall_curve, average_precision_score,
                             classification_report)

sns.set_style("whitegrid")
```

```
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")
print("Dataset loaded successfully!")
```

Output:

```
Dataset loaded successfully!
```

```
print("Shape of X (samples, features):", X.shape)
print("Shape of y:", y.shape)
print("\nTarget names:", data.target_names)
print("Target encoding: 0 = malignant, 1 = benign")
print("\nClass distribution:")
print(y.value_counts().rename({0: 'malignant (0)', 1: 'benign (1)'}))
print("\nMissing values:", X.isnull().sum().sum())
print("\nFeature names:")
for i, f in enumerate(data.feature_names, 1):
    print(f"{i:2d}. {f}")
```

Output:

```
Shape of X (samples, features): (569, 30)
Shape of y: (569,)

Target names: ['malignant' 'benign']
Target encoding: 0 = malignant, 1 = benign

Class distribution:
target
benign (1)       357
```

```
malignant (0)    212
Name: count, dtype: int64

Missing values: 0

Feature names:
 1. mean radius
 2. mean texture
 3. mean perimeter
 4. mean area
 5. mean smoothness
 6. mean compactness
 7. mean concavity
 8. mean concave points
 9. mean symmetry
10. mean fractal dimension
11. radius error
12. texture error
13. perimeter error
14. area error
15. smoothness error
16. compactness error
17. concavity error
18. concave points error
19. symmetry error
20. fractal dimension error
21. worst radius
22. worst texture
23. worst perimeter
24. worst area
25. worst smoothness
26. worst compactness
27. worst concavity
28. worst concave points
29. worst symmetry
30. worst fractal dimension
```

```
X.head()
```

Output:

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  mean compactness  mean concavity
mean concave points  mean symmetry  mean fractal dimension  radius error  texture error  perimeter error  area
error  smoothness error  compactness error  concavity error  concave points error  symmetry error  fractal
dimension error  worst radius  worst texture  worst perimeter  worst area  worst smoothness  worst compactness
worst concavity  worst concave points  worst symmetry  worst fractal dimension
0        17.99         10.38          122.80     1001.0          0.11840           0.27760          0.3001
0.14710          0.2419                0.07871        1.0950         0.9053            8.589      153.40
0.006399            0.04904          0.05373               0.01587         0.03003                 0.006193
25.38          17.33           184.60      2019.0            0.1622             0.6656           0.7119
0.2654          0.4601                 0.11890
1        20.57         17.77          132.90     1326.0          0.08474           0.07864          0.0869
0.07017          0.1812                0.05667        0.5435         0.7339            3.398       74.08
0.005225            0.01308          0.01860               0.01340         0.01389                 0.003532
24.99          23.41           158.80      1956.0            0.1238             0.1866           0.2416
0.1860          0.2750                 0.08902
2        19.69         21.25          130.00     1203.0          0.10960           0.15990          0.1974
0.12790          0.2069                0.05999        0.7456         0.7869            4.585       94.03
0.006150            0.04006          0.03832               0.02058         0.02250                 0.004571
23.57          25.53           152.50      1709.0            0.1444             0.4245           0.4504
0.2430          0.3613                 0.08758
```

```
3        11.42         20.38           77.58      386.1          0.14250           0.28390          0.2414
0.10520         0.2597                 0.09744        0.4956         1.1560            3.445       27.23
0.009110            0.07458          0.05661               0.01867         0.05963                 0.009208
14.91          26.50            98.87       567.7            0.2098             0.8663           0.6869
0.2575          0.6638                  0.17300
4        20.29         14.34          135.10     1297.0          0.10030           0.13280          0.1980
0.10430         0.1809                 0.05883        0.7572         0.7813            5.438       94.44
0.011490            0.02461          0.05688               0.01885         0.01756                 0.005115
22.54          16.67           152.20      1575.0            0.1374             0.2050           0.4000
0.1625          0.2364                  0.07678
```

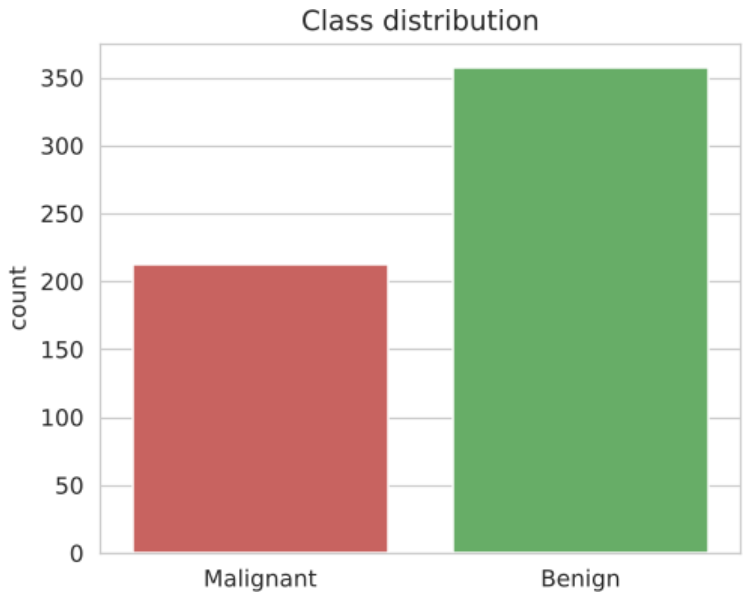
```
X.describe().T
```

Output:

```
                         count         mean          std          min          25%          50%          75%
max
mean radius              569.0    14.127292     3.524049     6.981000    11.700000    13.370000    15.780000
28.11000
mean texture             569.0    19.289649     4.301036     9.710000    16.170000    18.840000    21.800000
39.28000
mean perimeter           569.0    91.969033    24.298981    43.790000    75.170000    86.240000   104.100000
188.50000
mean area                569.0   654.889104   351.914129   143.500000   420.300000   551.100000   782.700000
2501.00000
mean smoothness          569.0     0.096360     0.014064     0.052630     0.086370     0.095870     0.105300
0.16340
mean compactness         569.0     0.104341     0.052813     0.019380     0.064920     0.092630     0.130400
0.34540
mean concavity           569.0     0.088799     0.079720     0.000000     0.029560     0.061540     0.130700
0.42680
mean concave points      569.0     0.048919     0.038803     0.000000     0.020310     0.033500     0.074000
0.20120
mean symmetry            569.0     0.181162     0.027414     0.106000     0.161900     0.179200     0.195700
0.30400
mean fractal dimension   569.0     0.062798     0.007060     0.049960     0.057700     0.061540     0.066120
0.09744
radius error             569.0     0.405172     0.277313     0.111500     0.232400     0.324200     0.478900
2.87300
texture error            569.0     1.216853     0.551648     0.360200     0.833900     1.108000     1.474000
4.88500
perimeter error          569.0     2.866059     2.021855     0.757000     1.606000     2.287000     3.357000
21.98000
area error               569.0    40.337079    45.491006     6.802000    17.850000    24.530000    45.190000
542.20000
smoothness error         569.0     0.007041     0.003003     0.001713     0.005169     0.006380     0.008146
0.03113
compactness error        569.0     0.025478     0.017908     0.002252     0.013080     0.020450     0.032450
0.13540
concavity error          569.0     0.031894     0.030186     0.000000     0.015090     0.025890     0.042050
0.39600
concave points error     569.0     0.011796     0.006170     0.000000     0.007638     0.010930     0.014710
0.05279
symmetry error           569.0     0.020542     0.008266     0.007882     0.015160     0.018730     0.023480
0.07895
fractal dimension error  569.0     0.003795     0.002646     0.000895     0.002248     0.003187     0.004558
0.02984
worst radius             569.0    16.269190     4.833242     7.930000    13.010000    14.970000    18.790000
36.04000
worst texture            569.0    25.677223     6.146258    12.020000    21.080000    25.410000    29.720000
49.54000
```

```
worst perimeter          569.0  107.261213   33.602542   50.410000   84.110000   97.660000   125.400000
251.20000
worst area               569.0  880.583128  569.356993  185.200000  515.300000  686.500000  1084.000000
4254.00000
worst smoothness         569.0    0.132369    0.022832    0.071170    0.116600    0.131300     0.146000
0.22260
worst compactness        569.0    0.254265    0.157336    0.027290    0.147200    0.211900     0.339100
1.05800
worst concavity          569.0    0.272188    0.208624    0.000000    0.114500    0.226700     0.382900
1.25200
worst concave points     569.0    0.114606    0.065732    0.000000    0.064930    0.099930     0.161400
0.29100
worst symmetry           569.0    0.290076    0.061867    0.156500    0.250400    0.282200     0.317900
0.66380
worst fractal dimension  569.0    0.083946    0.018061    0.055040    0.071460    0.080040     0.092080
0.20750
```

```
plt.figure(figsize=(5, 4))
sns.countplot(x=y.map({0: 'Malignant', 1: 'Benign'}), palette=["#d9534f", "#5cb85c"])
plt.title("Class distribution")
plt.xlabel("")
plt.show()
```




```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training set:", X_train.shape, "| Test set:", X_test.shape)
print("Train class ratio:\n", y_train.value_counts(normalize=True).round(3))
print("Test class ratio:\n", y_test.value_counts(normalize=True).round(3))
```

Output:

```
Training set: (455, 30) | Test set: (114, 30)
Train class ratio:
 target
1    0.626
0    0.374
Name: proportion, dtype: float64
Test class ratio:
 target
1    0.632
```

```
0    0.368
Name: proportion, dtype: float64
```

```
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, random_state=42))
model.fit(X_train, y_train)
print("Model trained.")
print("Training accuracy:", round(model.score(X_train, y_train), 4))
print("Test accuracy    :", round(model.score(X_test, y_test), 4))
```

Output:

```
Model trained.
Training accuracy: 0.989
Test accuracy    : 0.9825
```

```
y_pred = model.predict(X_test)                  # uses threshold 0.5
y_prob = model.predict_proba(X_test)[:, 1]      # probability of class 1 (benign)

pd.DataFrame({"Actual": y_test.values[:10],
              "Predicted": y_pred[:10],
              "P(class=1)": y_prob[:10].round(4)})
```

Output:

	Actual	Predicted	P(class=1)
0	0	0	0.0000
1	1	1	1.0000
2	0	0	0.0064
3	1	1	0.5335
4	0	0	0.0000
5	1	1	0.9922
6	1	1	1.0000
7	0	0	0.0000
8	0	0	0.0001
9	0	0	0.0000

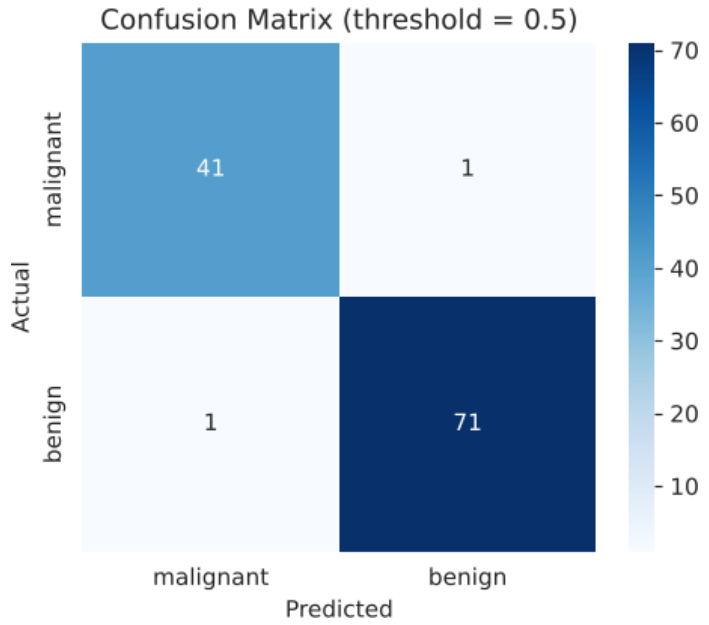
```
cm = confusion_matrix(y_test, y_pred)
print(cm)

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.title("Confusion Matrix (threshold = 0.5)")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
```

Output:

```
[[41  1]
 [ 1 71]]
TN=41, FP=1, FN=1, TP=71
```




```
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred):.4f}")
print("\nClassification report:\n")
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

Output:

```
Accuracy : 0.9825
Precision: 0.9861
Recall   : 0.9861
F1-Score : 0.9861

Classification report:

              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

```
thresholds = [0.5, 0.9, 0.3]
results = []
cms = {}

for t in thresholds:
    y_t = (y_prob >= t).astype(int)
    cms[t] = confusion_matrix(y_test, y_t)
    tn, fp, fn, tp = cms[t].ravel()
    results.append({
        "Threshold": t,
        "TN": tn, "FP": fp, "FN": fn, "TP": tp,
        "Accuracy": accuracy_score(y_test, y_t),
        "Precision": precision_score(y_test, y_t, zero_division=0),
        "Recall": recall_score(y_test, y_t),
        "F1-Score": f1_score(y_test, y_t),
```

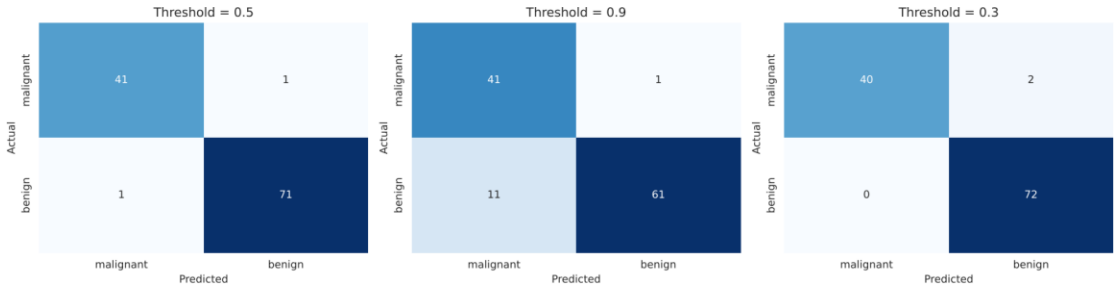
```
    })

results_df = pd.DataFrame(results).set_index("Threshold").round(4)
results_df
```

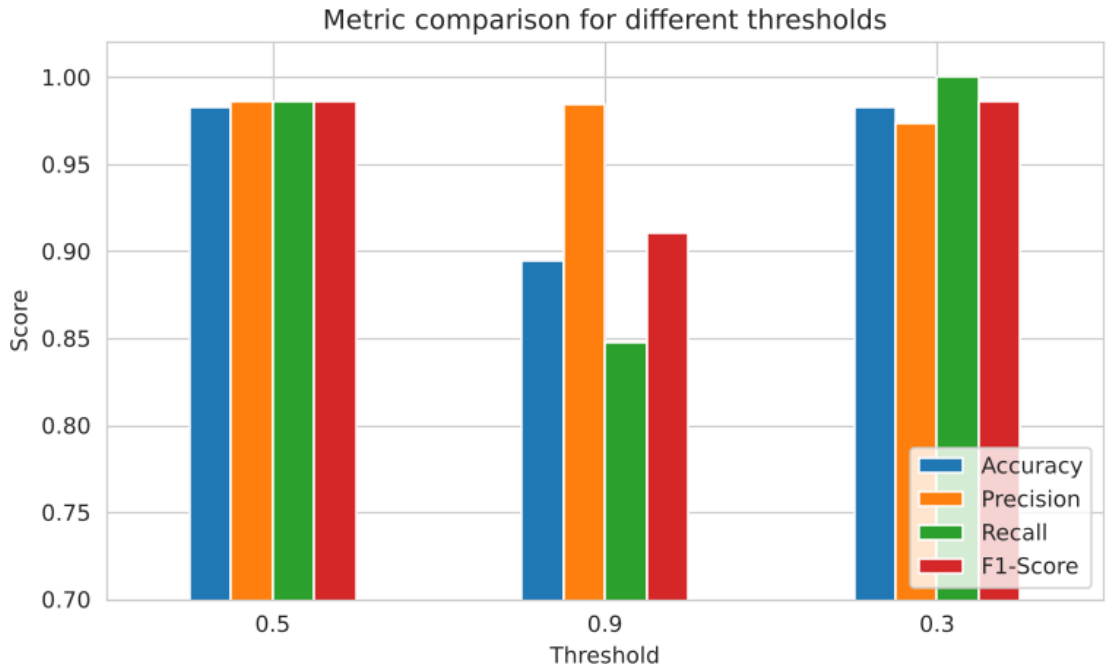
Output:

```
           TN  FP  FN  TP  Accuracy  Precision  Recall  F1-Score
Threshold
0.5        41   1   1  71    0.9825     0.9861  0.9861    0.9861
0.9        41   1  11  61    0.8947     0.9839  0.8472    0.9104
0.3        40   2   0  72    0.9825     0.9730  1.0000    0.9863
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, t in zip(axes, thresholds):
    sns.heatmap(cms[t], annot=True, fmt="d", cmap="Blues", ax=ax, cbar=False,
                xticklabels=data.target_names, yticklabels=data.target_names)
    ax.set_title(f"Threshold = {t}")
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout()
plt.show()
```




```
# Compare metrics across thresholds
results_df[["Accuracy", "Precision", "Recall", "F1-Score"]].plot(kind="bar", figsize=(8, 4.5), rot=0)
plt.title("Metric comparison for different thresholds")
plt.ylabel("Score")
plt.ylim(0.7, 1.02)
plt.legend(loc="lower right")
plt.show()
```



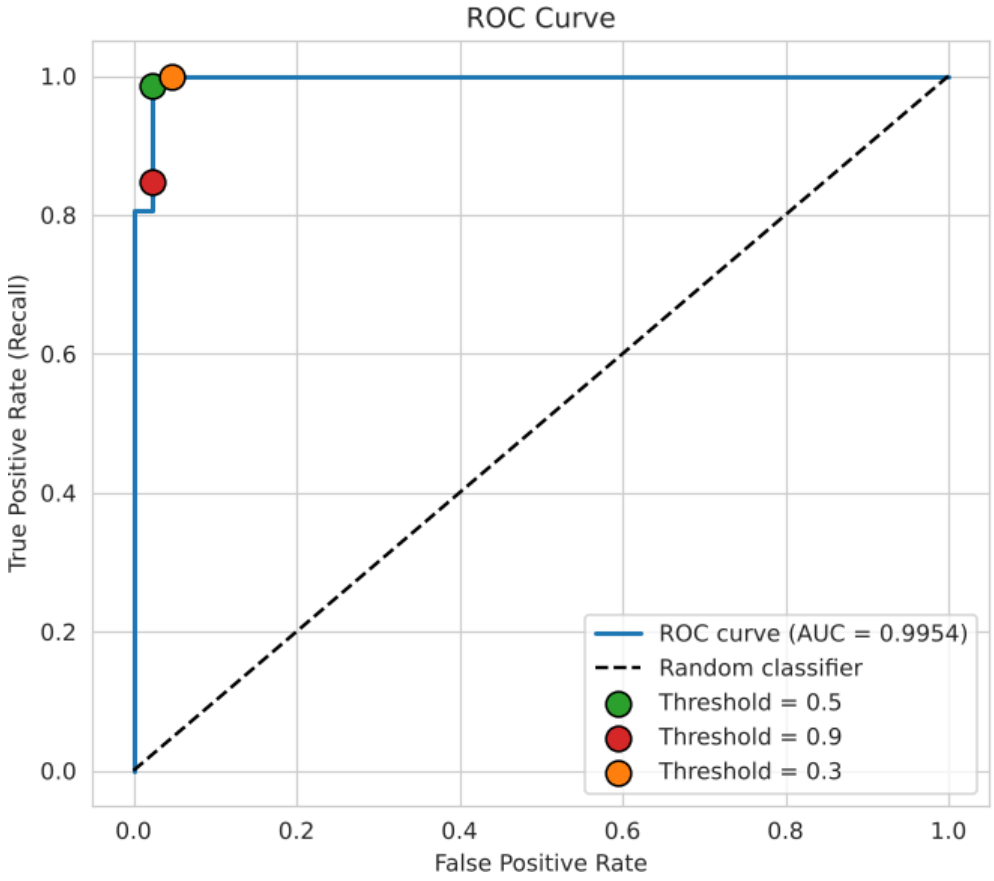

```
fpr, tpr, roc_thr = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

colors = {0.5: "tab:green", 0.9: "tab:red", 0.3: "tab:orange"}

plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color="tab:blue", lw=2, label=f"ROC curve (AUC = {auc:.4f})")
plt.plot([0, 1], [0, 1], "k--", label="Random classifier")

for t in thresholds:
    y_t = (y_prob >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, y_t).ravel()
    plt.scatter(fp / (fp + tn), tp / (tp + fn), s=120, color=colors[t],
                edgecolor="k", zorder=5, label=f"Threshold = {t}")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate (Recall)")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.show()
```

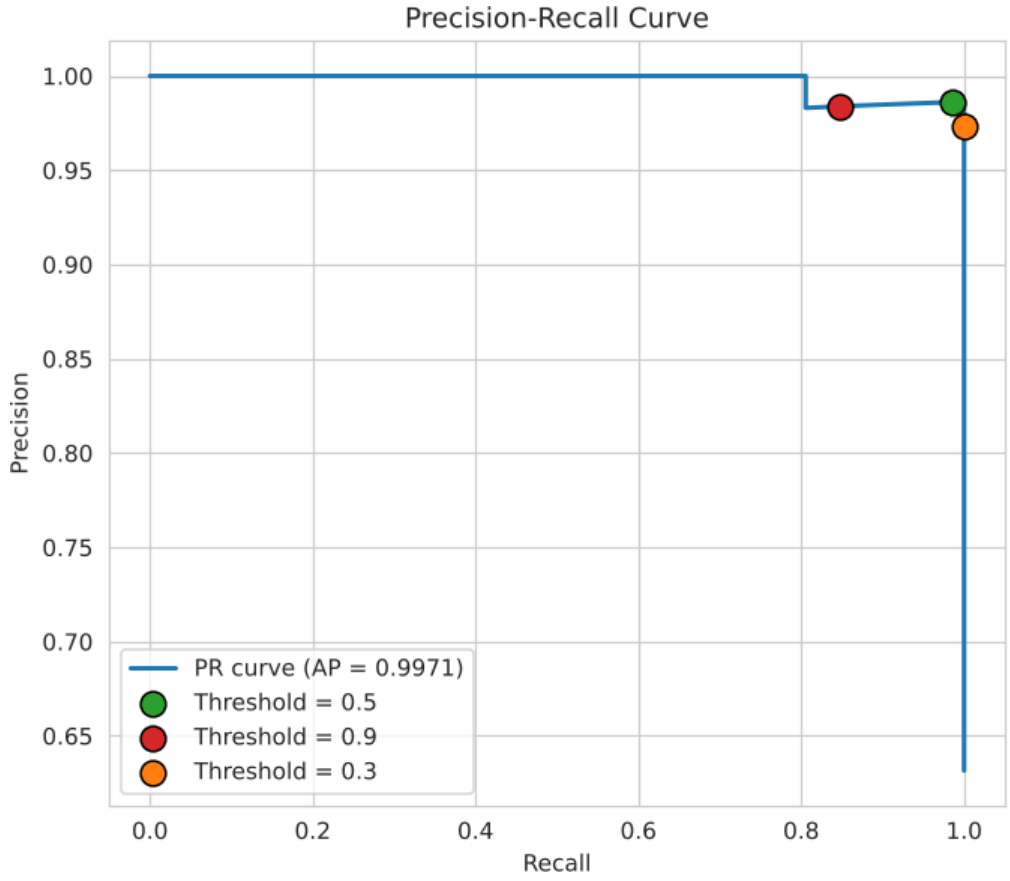



```
prec, rec, pr_thr = precision_recall_curve(y_test, y_prob)
ap = average_precision_score(y_test, y_prob)

plt.figure(figsize=(7, 6))
plt.plot(rec, prec, color="tab:blue", lw=2, label=f"PR curve (AP = {ap:.4f})")

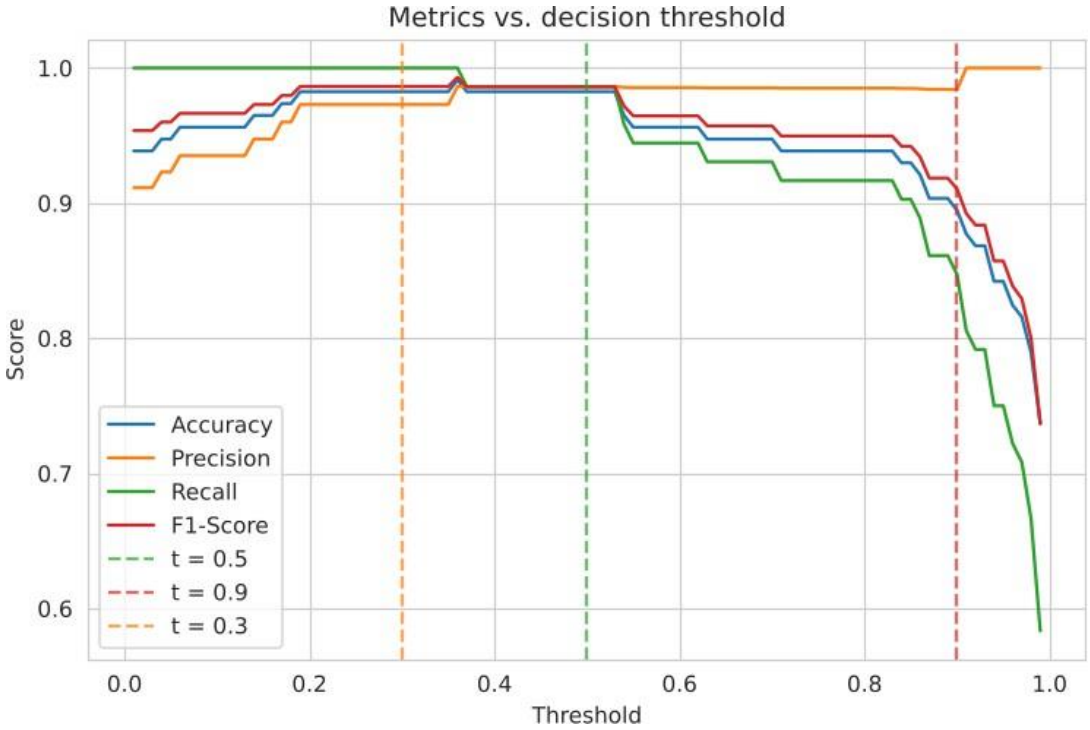
for t in thresholds:
    y_t = (y_prob >= t).astype(int)
    plt.scatter(recall_score(y_test, y_t), precision_score(y_test, y_t, zero_division=0),
                s=120, color=colors[t], edgecolor="k", zorder=5, label=f"Threshold = {t}")

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend(loc="lower left")
plt.show()
```




```
# Both curves side by side + metrics vs threshold
ths = np.linspace(0.01, 0.99, 99)
p_list, r_list, f_list, a_list = [], [], [], []
for t in ths:
    yt = (y_prob >= t).astype(int)
    p_list.append(precision_score(y_test, yt, zero_division=0))
    r_list.append(recall_score(y_test, yt))
    f_list.append(f1_score(y_test, yt))
    a_list.append(accuracy_score(y_test, yt))

plt.figure(figsize=(8, 5))
plt.plot(ths, a_list, label="Accuracy")
plt.plot(ths, p_list, label="Precision")
plt.plot(ths, r_list, label="Recall")
plt.plot(ths, f_list, label="F1-Score")
for t in thresholds:
    plt.axvline(t, color=colors[t], ls="--", alpha=0.7, label=f"t = {t}")
plt.xlabel("Threshold"); plt.ylabel("Score")
plt.title("Metrics vs. decision threshold")
plt.legend()
plt.show()
```


Metrics vs. decision threshold
Score
Threshold
Accuracy
Precision
Recall
F1-Score
t = 0.5
t = 0.9
t = 0.3
1.0
0.9
0.8
0.7
0.6
0.0
0.2
0.4
0.6
0.8
1.0

Practical No.5

Decision Tree and Random Forest for Classification

Aim:

Implement Decision Tree and Random Forest classifiers on the Breast Cancer Wisconsin dataset and compare their performance.

Theory

Decision Tree and Random Forest are supervised machine learning algorithms used for classification and prediction. In this practical, both algorithms are implemented on the Breast Cancer Wisconsin dataset to classify tumors as malignant (cancerous) or benign (non-cancerous) and compare their performance.

1. Decision Tree Classifier

A Decision Tree is a supervised machine learning algorithm that makes predictions using a tree-like structure. It divides the dataset into smaller groups based on feature values until a final class prediction is obtained.

The main components of a Decision Tree are:

- Root Node: The starting point of the tree.
- Decision Node: A node that applies a condition to split the data.
- Branch: The outcome of a decision.
- Leaf Node: The final predicted class.

Decision Trees select splits using criteria such as Gini impurity or entropy.

Advantages:

- Easy to understand and interpret.
- Requires little feature preprocessing.
- Can capture nonlinear relationships.

Limitations:

- May overfit the training data.
- Small changes in data can produce a different tree.

2. Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction performance. Each tree is trained using a random sample of the training data and considers a random subset of features when selecting splits.

For classification, the final prediction is generally determined by majority voting among the individual trees.

For example, if 8 out of 10 trees predict a tumor as benign, the Random Forest predicts the tumor as benign.

Advantages:

- Usually reduces overfitting compared with a single tree.
- Often provides better generalization.
- Handles complex relationships between features.

- Can estimate feature importance.

Limitations:

- More computationally expensive than a single Decision Tree.
- More difficult to interpret because it contains many trees.

3. Breast Cancer Wisconsin Dataset

The Breast Cancer Wisconsin dataset contains measurements of cell nuclei obtained from breast mass samples. It includes features such as:

- Radius
- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity

The target variable contains two classes:

- Malignant: Cancerous tumor.
- Benign: Non-cancerous tumor.

The dataset is used to train both classifiers and evaluate their ability to predict tumor classes.

Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             roc_curve, auc, ConfusionMatrixDisplay)

import warnings
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
```

```
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')

print("Shape of features:", X.shape)
print("Classes:", dict(zip(data.target_names, [0, 1])))
print("\nClass distribution:")
print(y.value_counts())
X.head()
```

Output:

```
Shape of features: (569, 30)
Classes: {np.str_('malignant'): 0, np.str_('benign'): 1}

Class distribution:
target
1    357
0    212
Name: count, dtype: int64
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst radius  worst texture   worst perimeter  \
0                 0.07871  ...         25.38          17.33           184.60
1                 0.05667  ...         24.99          23.41           158.80
2                 0.05999  ...         23.57          25.53           152.50
3                 0.09744  ...         14.91          26.50            98.87
4                 0.05883  ...         22.54          16.67           152.20

   worst area  worst smoothness  worst compactness   worst concavity  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   worst concave points  worst symmetry  worst fractal dimension
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 30 columns]
```

```
print("Missing values:", X.isnull().sum().sum())
X.describe().T.head(10)
```

Output:

```
Missing values: 0
```

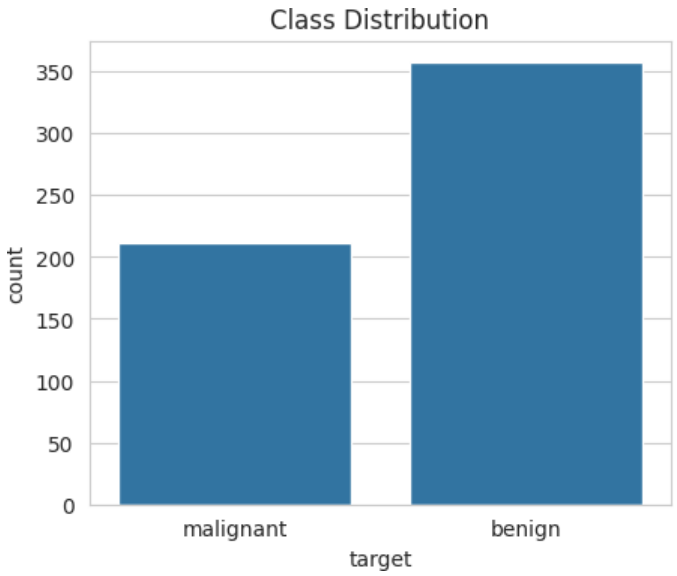
```
                count        mean         std        min        25%  \
mean radius     569.0   14.127292    3.524049    6.98100   11.70000
mean texture    569.0   19.289649    4.301036    9.71000   16.17000
```

```
mean perimeter           569.0   91.969033   24.298981   43.79000   75.17000
mean area                569.0  654.889104  351.914129  143.50000  420.30000
mean smoothness          569.0    0.096360    0.014064    0.05263    0.08637
mean compactness         569.0    0.104341    0.052813    0.01938    0.06492
mean concavity           569.0    0.088799    0.079720    0.00000    0.02956
mean concave points      569.0    0.048919    0.038803    0.00000    0.02031
mean symmetry            569.0    0.181162    0.027414    0.10600    0.16190
mean fractal dimension   569.0    0.062798    0.007060    0.04996    0.05770

                              50%         75%         max
mean radius               13.37000    15.78000    28.11000
mean texture              18.84000    21.80000    39.28000
mean perimeter            86.24000   104.10000   188.50000
mean area                551.10000   782.70000  2501.00000
mean smoothness            0.09587     0.10530     0.16340
mean compactness           0.09263     0.13040     0.34540
mean concavity             0.06154     0.13070     0.42680
mean concave points        0.03350     0.07400     0.20120
mean symmetry              0.17920     0.19570     0.30400
mean fractal dimension     0.06154     0.06612     0.09744
```

```
plt.figure(figsize=(5, 4))
sns.countplot(x=y.map({0: 'malignant', 1: 'benign'}))
plt.title("Class Distribution")
plt.show()
```

Output:




```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Training set:", X_train.shape)
print("Testing set :", X_test.shape)
```

Output:

```
Training set: (455, 30)
Testing set : (114, 30)
```

```
dt = DecisionTreeClassifier(criterion='gini', random_state=42)
```

```
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
print("Decision Tree (unpruned)")
print("Train accuracy:", accuracy_score(y_train, dt.predict(X_train)))
print("Test accuracy :", accuracy_score(y_test, y_pred_dt))
print("Tree depth    :", dt.get_depth())
print("Leaves        :", dt.get_n_leaves())
```

Output:

```
Decision Tree (unpruned)
Train accuracy: 1.0
Test accuracy : 0.9122807017543859
Tree depth    : 7
Leaves        : 19
```

```
dt_pruned = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_pruned.fit(X_train, y_train)
y_pred_dtp = dt_pruned.predict(X_test)

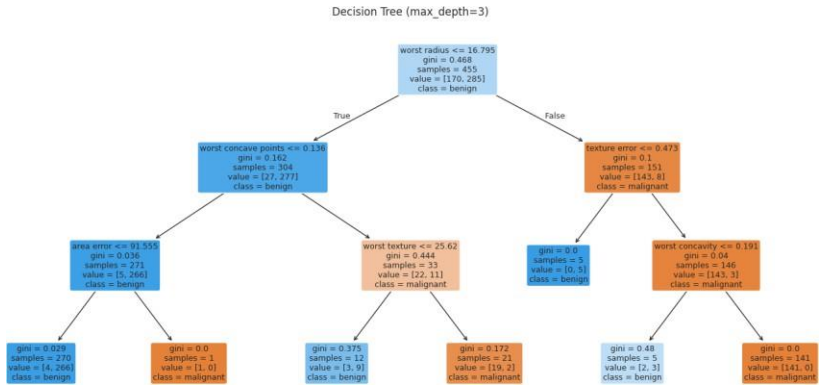
print("Pruned Decision Tree (max_depth=3)")
print("Train accuracy:", accuracy_score(y_train, dt_pruned.predict(X_train)))
print("Test accuracy :", accuracy_score(y_test, y_pred_dtp))
```

Output:

```
Pruned Decision Tree (max_depth=3)
Train accuracy: 0.9758241758241758
Test accuracy : 0.9385964912280702
```

```
plt.figure(figsize=(18, 8))
plot_tree(dt_pruned, feature_names=data.feature_names,
          class_names=data.target_names, filled=True, rounded=True, fontsize=9)
plt.title("Decision Tree (max_depth=3)")
plt.show()
```

Output:




```
print(export_text(dt_pruned, feature_names=list(data.feature_names)))
```

Output:

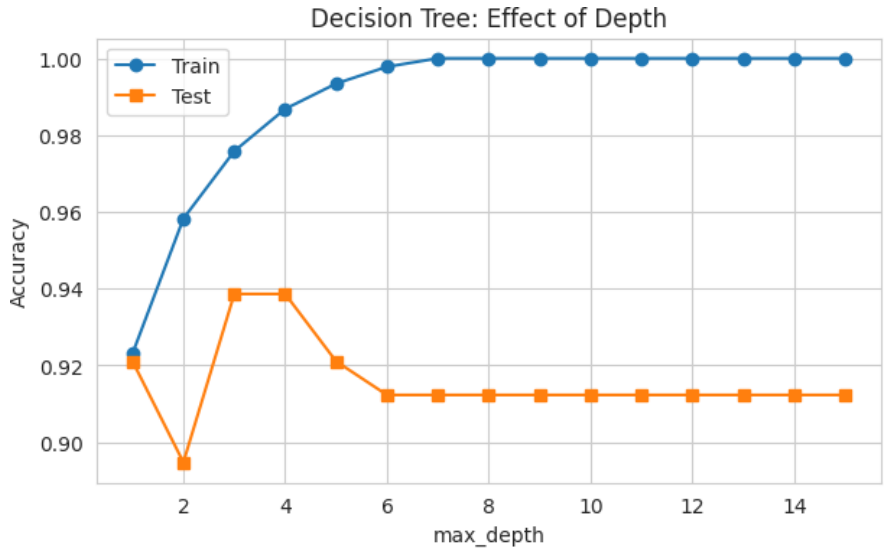
```
|--- worst radius <= 16.80
|   |--- worst concave points <= 0.14
|   |   |--- area error <= 91.56
```

```
|   |   |   |--- class: 1
|   |   |--- area error >  91.56
|   |   |   |--- class: 0
|   |--- worst concave points >  0.14
|   |   |--- worst texture <= 25.62
|   |   |   |--- class: 1
|   |   |--- worst texture >  25.62
|   |   |   |--- class: 0
|--- worst radius >  16.80
|   |--- texture error <= 0.47
|   |   |--- class: 1
|   |--- texture error >  0.47
|   |   |--- worst concavity <= 0.19
|   |   |   |--- class: 1
|   |   |--- worst concavity >  0.19
|   |   |   |--- class: 0
```

```
# Effect of max_depth
depths = range(1, 16)
train_acc, test_acc = [], []
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
    train_acc.append(m.score(X_train, y_train))
    test_acc.append(m.score(X_test, y_test))

plt.figure(figsize=(7, 4))
plt.plot(depths, train_acc, 'o-', label='Train')
plt.plot(depths, test_acc, 's-', label='Test')
plt.xlabel('max_depth'); plt.ylabel('Accuracy')
plt.title('Decision Tree: Effect of Depth'); plt.legend()
plt.show()
```

Output:




```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print("Random Forest")
print("Train accuracy:", accuracy_score(y_train, rf.predict(X_train)))
print("Test accuracy :", accuracy_score(y_test, y_pred_rf))
```

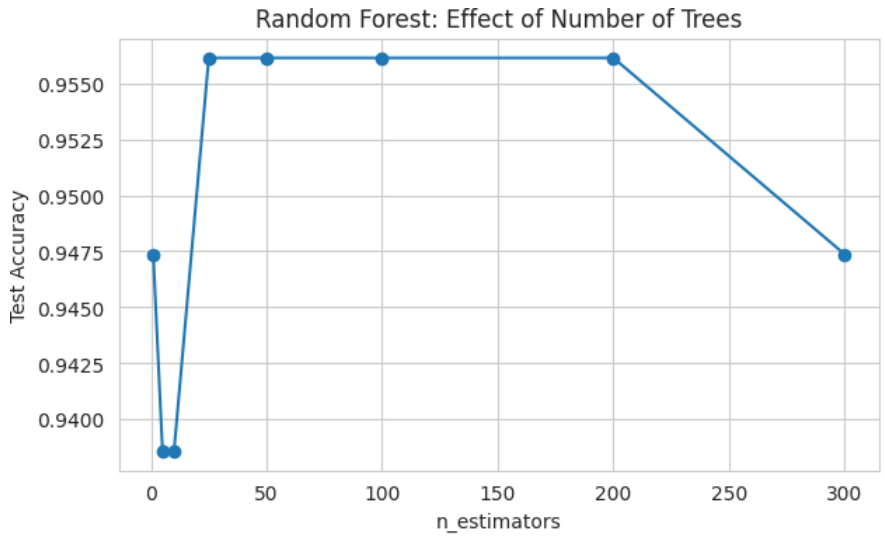
Output:

```
Random Forest
Train accuracy: 1.0
Test accuracy : 0.956140350877193
```

```
# Effect of number of trees
n_trees = [1, 5, 10, 25, 50, 100, 200, 300]
scores = []
for n in n_trees:
    m = RandomForestClassifier(n_estimators=n, random_state=42).fit(X_train, y_train)
    scores.append(m.score(X_test, y_test))

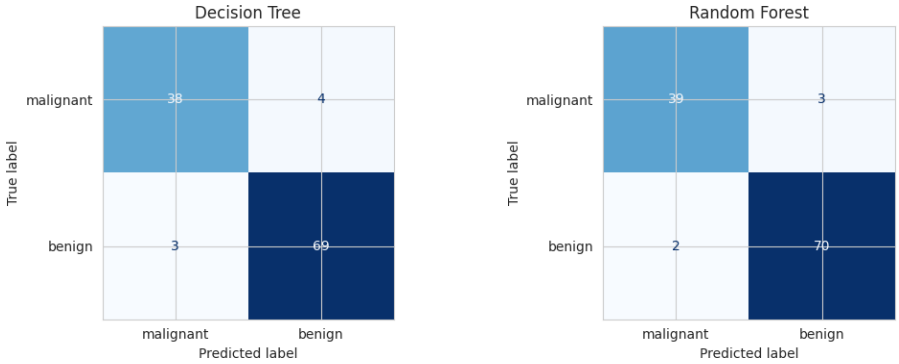
plt.figure(figsize=(7, 4))
plt.plot(n_trees, scores, 'o-')
plt.xlabel('n_estimators'); plt.ylabel('Test Accuracy')
plt.title('Random Forest: Effect of Number of Trees')
plt.show()
```

Output:




```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (name, pred) in zip(axes, [('Decision Tree', y_pred_dtp), ('Random Forest',
y_pred_rf)]):
    ConfusionMatrixDisplay(confusion_matrix(y_test, pred),
                           display_labels=data.target_names).plot(ax=ax, cmap='Blues',
colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.show()
```

Output:




```
print("=== Decision Tree (max_depth=3) ===")
print(classification_report(y_test, y_pred_dtp, target_names=data.target_names))
print("=== Random Forest ===")
print(classification_report(y_test, y_pred_rf, target_names=data.target_names))
```

Output:

```
=== Decision Tree (max_depth=3) ===
              precision    recall  f1-score   support

   malignant       0.93      0.90      0.92        42
      benign       0.95      0.96      0.95        72

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114

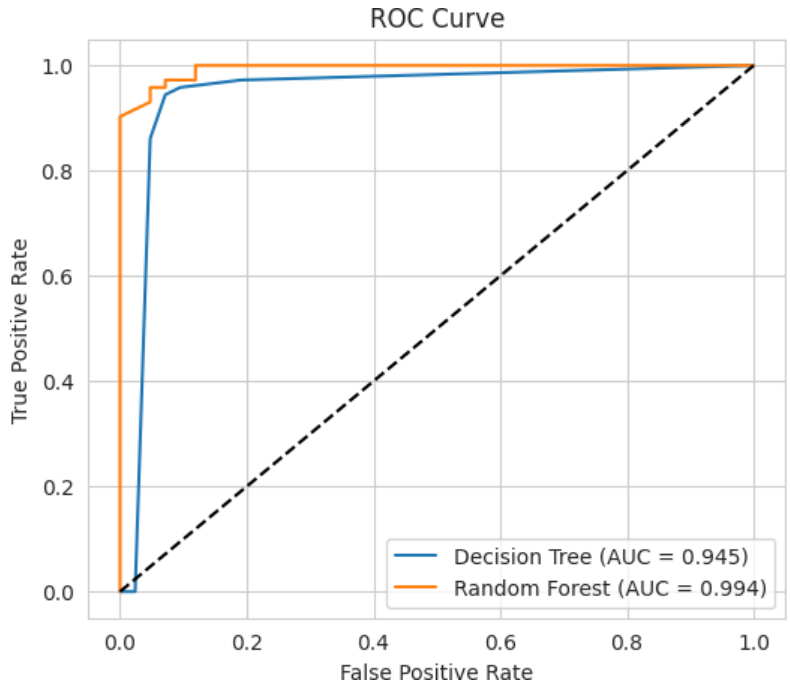
=== Random Forest ===
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

```
plt.figure(figsize=(6, 5))
for name, model in [('Decision Tree', dt_pruned), ('Random Forest', rf)]:
    prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, prob)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc(fpr, tpr):.3f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve'); plt.legend()
plt.show()
```

Output:




```
for name, model in [('Decision Tree', DecisionTreeClassifier(max_depth=3,
random_state=42)),
                    ('Random Forest', RandomForestClassifier(n_estimators=100,
random_state=42))]:
    cv = cross_val_score(model, X, y, cv=5)
    print(f"{name:15s} CV accuracy: {cv.mean():.4f} (+/- {cv.std():.4f})")
```

Output:

```
Decision Tree   CV accuracy: 0.9191 (+/- 0.0246)
```

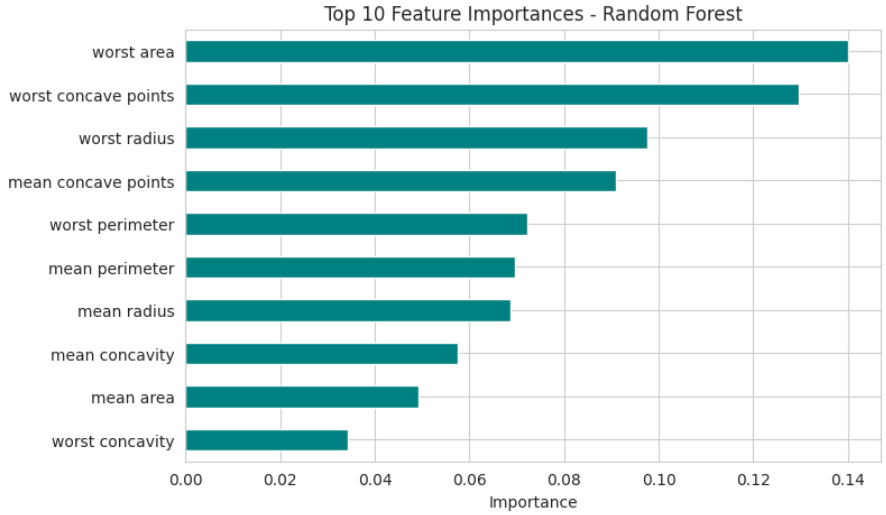
```
Random Forest   CV accuracy: 0.9561 (+/- 0.0228)
```

```
imp = pd.Series(rf.feature_importances_,
index=data.feature_names).sort_values(ascending=False)
print(imp.head(10))

plt.figure(figsize=(8, 5))
imp.head(10).sort_values().plot(kind='barh', color='teal')
plt.title('Top 10 Feature Importances - Random Forest')
plt.xlabel('Importance')
plt.show()
```

Output:

```
worst area              0.140016
worst concave points    0.129530
worst radius            0.097696
mean concave points     0.090885
worst perimeter         0.072226
mean perimeter          0.069574
mean radius             0.068676
mean concavity          0.057638
mean area               0.049172
worst concavity         0.034340
dtype: float64
```




```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid,
                    cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score  :", round(grid.best_score_, 4))

best_rf = grid.best_estimator_
print("Test accuracy (tuned RF):", round(accuracy_score(y_test, best_rf.predict(X_test)),
4))
```

Output:

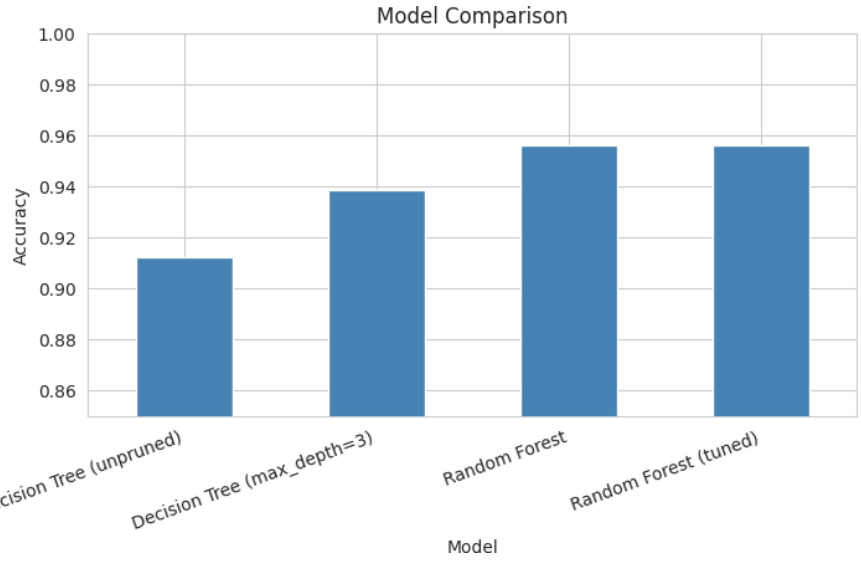
```
Best parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 200}
Best CV score  : 0.9604
Test accuracy (tuned RF): 0.9561
```

```
results = pd.DataFrame({
    'Model': ['Decision Tree (unpruned)', 'Decision Tree (max_depth=3)', 'Random Forest',
'Random Forest (tuned)'],
    'Test Accuracy': [accuracy_score(y_test, y_pred_dt), accuracy_score(y_test,
y_pred_dtp),
                      accuracy_score(y_test, y_pred_rf), accuracy_score(y_test,
best_rf.predict(X_test))]
}).round(4)
display(results)

results.plot(x='Model', y='Test Accuracy', kind='bar', legend=False, figsize=(8, 4),
color='steelblue')
plt.ylim(0.85, 1.0); plt.ylabel('Accuracy'); plt.xticks(rotation=20, ha='right')
plt.title('Model Comparison')
plt.show()
```

Output:

```
                          Model  Test Accuracy
0     Decision Tree (unpruned)         0.9123
1  Decision Tree (max_depth=3)         0.9386
2                Random Forest         0.9561
3        Random Forest (tuned)         0.9561
```




```
sample = X_test.iloc[[0]]
print("Actual    :", data.target_names[y_test.iloc[0]])
print("Decision Tree :", data.target_names[dt_pruned.predict(sample)[0]])
print("Random Forest :", data.target_names[rf.predict(sample)[0]])
print("RF probabilities:", dict(zip(data.target_names,
rf.predict_proba(sample)[0].round(3))))
```

Output:

```
Actual    : malignant
Decision Tree : malignant
Random Forest : malignant
RF probabilities: {np.str_('malignant'): np.float64(1.0), np.str_('benign'):
np.float64(0.0)}
```

Practical No-6

K-Nearest Neighbor (KNN)

Aim:

To implement the K-Nearest Neighbor (KNN) algorithm on the make_moons dataset, select the best k using 5-fold cross validation, and visualize the confusion matrix and decision boundary.

Theory

K-Nearest Neighbor (KNN) is a supervised machine learning algorithm used for classification and regression. It is a simple, non-parametric, and instance-based learning algorithm that predicts the class of a new data point based on the classes of its nearest neighboring points.

In this practical, the KNN algorithm is implemented on the make_moons dataset to classify data points into two classes, select the best value of \(k\) using 5-fold cross-validation, and visualize the confusion matrix and decision boundary.

1. K-Nearest Neighbor Algorithm

KNN classifies a new data point by calculating its distance from existing data points and identifying the \(k\) nearest neighbors. The predicted class is generally determined by majority voting among these neighbors.

The Euclidean distance between two points is calculated as:

represent two data points.

Steps of the KNN algorithm:

1. Choose the value of \(k\).
2. Calculate the distance between the new point and all training points.
3. Select the \(k\) nearest neighbors.
4. Count the class labels of these neighbors.
5. Assign the majority class to the new point.

2. Make Moons Dataset

The make_moons dataset is a synthetic dataset provided by Scikit-learn. It contains two interleaving crescent-shaped groups of data points.

- It is used to demonstrate nonlinear classification.
- It contains two features, usually represented
- It has two target classes, 0 and 1.
- Noise can be added to make the classification task more challenging.

This dataset helps demonstrate how KNN can classify data points with nonlinear decision boundaries.

3. Choosing the Best Value of K

The value of \(k\) determines the number of neighbors considered for classification.

- Small \(k\): The model may be sensitive to noise and overfit the data.
- Large \(k\): The model may oversimplify the decision boundary and underfit the data.

The best value of \(k\) is selected by testing different values and comparing their cross-validation scores.

4. 5-Fold Cross-Validation

Cross-validation is a technique used to estimate model performance on unseen data.

In 5-fold cross-validation, the training dataset is divided into five approximately equal parts. The model is trained five times, using four parts for training and the remaining part for validation in each iteration.

The average validation accuracy is calculated as:

\[\text{Mean Accuracy}=\frac{1}{5}\sum_{i=1}^{5}A_i \]

Where \(A_i\) represents the accuracy obtained in each fold.

The value of \(k\) with the best average cross-validation score is selected. The test dataset should remain separate until final evaluation.

5. Confusion Matrix

A confusion matrix evaluates the classification performance of the KNN model by comparing actual and predicted class labels.

	Predicted Class 0	Predicted Class 1
Actual Class 0	True Negative (TN)	False Positive (FP)
Actual Class 1	False Negative (FN)	True Positive (TP)

It helps identify correct predictions and classification errors for both classes.

Code and Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import ListedColormap

from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)
```

```
sns.set_style("whitegrid")
```

```
X, y = make_moons(n_samples=500, noise=0.3, random_state=42)
print("X shape:", X.shape, "| y shape:", y.shape)
print("Class counts:", dict(zip(*np.unique(y, return_counts=True))))
pd.DataFrame(X, columns=["Feature 1", "Feature 2"]).assign(Target=y).head()
```

Output:

```
X shape: (500, 2) | y shape: (500,)
Class counts: {np.int64(0): np.int64(250), np.int64(1): np.int64(250)}
```

Output:

```
   Feature 1  Feature 2  Target
0   0.831039  -0.258749       1
1   1.185064   0.920387       0
2   1.164022  -0.455526       1
3  -0.023656   1.086288       0
4   0.480503   1.509424       0
```

```
plt.figure(figsize=(7, 5))
plt.scatter(X[y == 0, 0], X[y == 0, 1], c="tab:blue", edgecolor="k", label="Class 0")
plt.scatter(X[y == 1, 0], X[y == 1, 1], c="tab:orange", edgecolor="k", label="Class 1")
plt.xlabel("Feature 1"); plt.ylabel("Feature 2")
plt.title("make_moons dataset")
plt.legend()
plt.show()
```




```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_test = scaler.transform(X_test_raw)
```

```
print("Train:", X_train.shape, "| Test:", X_test.shape)
print("Train mean (approx):", X_train.mean(axis=0).round(3), "| std:", X_train.std(axis=0).round(3))
```

Output:

```
Train: (400, 2) | Test: (100, 2)
Train mean (approx): [ 0. -0.] | std: [1. 1.]
```

```
def evaluate(y_true, y_pred, title=""):
    print(title)
    print(f"Accuracy : {accuracy_score(y_true, y_pred):.4f}")
    print(f"Precision: {precision_score(y_true, y_pred):.4f}")
    print(f"Recall   : {recall_score(y_true, y_pred):.4f}")
    print(f"F1-Score : {f1_score(y_true, y_pred):.4f}")

knn3 = KNeighborsClassifier(n_neighbors=3)
knn3.fit(X_train, y_train)
y_pred3 = knn3.predict(X_test)

evaluate(y_test, y_pred3, "Results for k = 3")
print("\n", classification_report(y_test, y_pred3))
```

Output:

```
Results for k = 3
Accuracy : 0.9100
Precision: 0.9184
Recall   : 0.9000
F1-Score : 0.9091

               precision    recall  f1-score   support

           0       0.90      0.92      0.91        50
           1       0.92      0.90      0.91        50

    accuracy                           0.91       100
   macro avg       0.91      0.91      0.91       100
weighted avg       0.91      0.91      0.91       100
```

```
k_values = range(1, 21)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_mean, cv_std = [], []
for k in k_values:
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train,
                             cv=cv, scoring="accuracy")
    cv_mean.append(scores.mean())
    cv_std.append(scores.std())

cv_results = pd.DataFrame({"k": list(k_values), "Mean CV Accuracy": cv_mean, "Std": cv_std}).round(4)
cv_results
```

Output:

```
    k  Mean CV Accuracy     Std
0   1            0.8800  0.0203
1   2            0.8950  0.0257
2   3            0.9050  0.0170
3   4            0.8975  0.0184
4   5            0.9025  0.0146
5   6            0.8950  0.0203
```

```
6    7            0.9050  0.0232
7    8            0.9000  0.0209
8    9            0.9050  0.0281
9   10            0.9075  0.0257
10  11            0.9200  0.0257
11  12            0.9125  0.0194
12  13            0.9175  0.0269
13  14            0.9250  0.0112
14  15            0.9250  0.0262
15  16            0.9200  0.0332
16  17            0.9200  0.0341
17  18            0.9225  0.0184
18  19            0.9175  0.0269
19  20            0.9125  0.0250
```

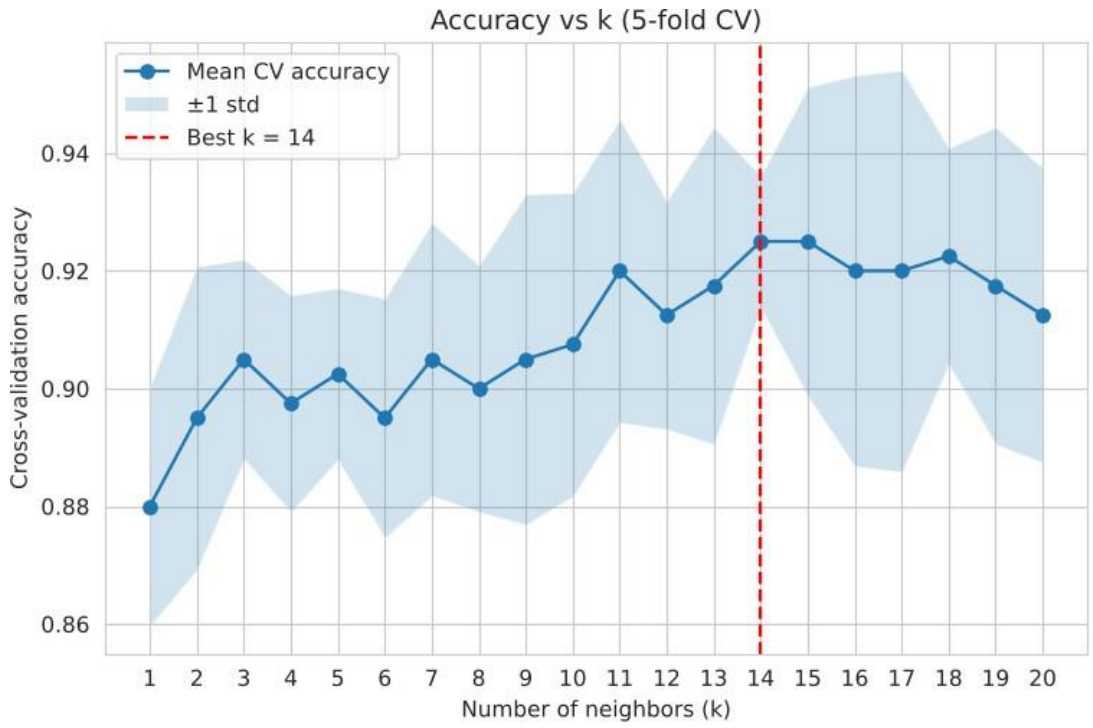
```
cv_mean = np.array(cv_mean); cv_std = np.array(cv_std)
best_k = list(k_values)[int(np.argmax(cv_mean))]   # first k with the highest accuracy
best_acc = cv_mean.max()

plt.figure(figsize=(8, 5))
plt.plot(k_values, cv_mean, marker="o", label="Mean CV accuracy")
plt.fill_between(k_values, cv_mean - cv_std, cv_mean + cv_std, alpha=0.2, label="±1 std")
plt.axvline(best_k, color="r", ls="--", label=f"Best k = {best_k}")
plt.xticks(list(k_values))
plt.xlabel("Number of neighbors (k)")
plt.ylabel("Cross-validation accuracy")
plt.title("Accuracy vs k (5-fold CV)")
plt.legend()
plt.show()

print(f"Best k = {best_k} with mean CV accuracy = {best_acc:.4f}")
```

Output:

```
Best k = 14 with mean CV accuracy = 0.9250
```



```
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
y_pred_best = knn_best.predict(X_test)

evaluate(y_test, y_pred_best, f"Results for best k = {best_k}")
print("\n", classification_report(y_test, y_pred_best))

comparison = pd.DataFrame({
    "k": [3, best_k],
    "Accuracy": [accuracy_score(y_test, y_pred3), accuracy_score(y_test, y_pred_best)],
    "Precision": [precision_score(y_test, y_pred3), precision_score(y_test, y_pred_best)],
    "Recall": [recall_score(y_test, y_pred3), recall_score(y_test, y_pred_best)],
    "F1-Score": [f1_score(y_test, y_pred3), f1_score(y_test, y_pred_best)],
}).round(4)
comparison
```

Output:

```
Results for best k = 14
Accuracy : 0.9200
Precision: 0.9375
Recall   : 0.9000
F1-Score : 0.9184

               precision    recall  f1-score   support

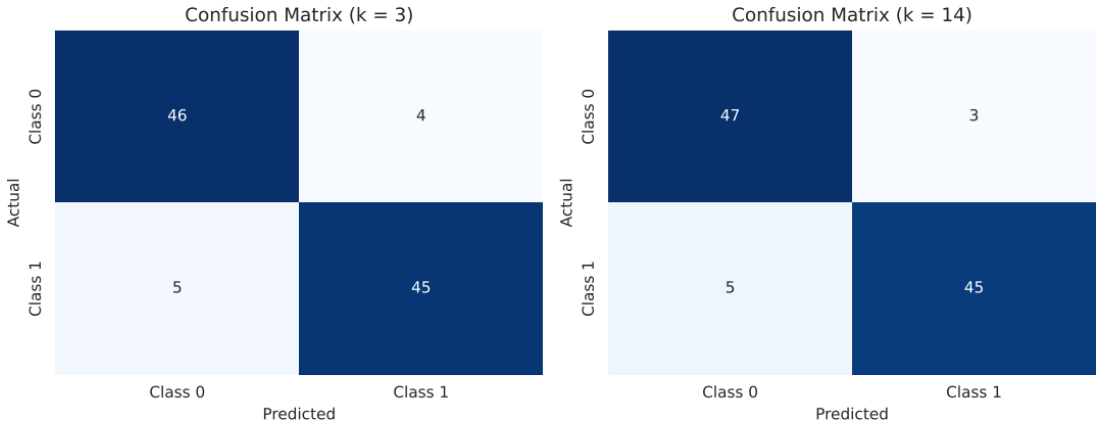
           0       0.90      0.94      0.92        50
           1       0.94      0.90      0.92        50

    accuracy                           0.92       100
   macro avg       0.92      0.92      0.92       100
weighted avg       0.92      0.92      0.92       100
```

Output:

```
    k  Accuracy  Precision  Recall  F1-Score
0   3      0.91     0.9184     0.9    0.9091
1  14      0.92     0.9375     0.9    0.9184
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, pred, k in zip(axes, [y_pred3, y_pred_best], [3, best_k]):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["Class 0", "Class 1"], yticklabels=["Class 0", "Class 1"])
    ax.set_title(f"Confusion Matrix (k = {k})")
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout()
plt.show()
```

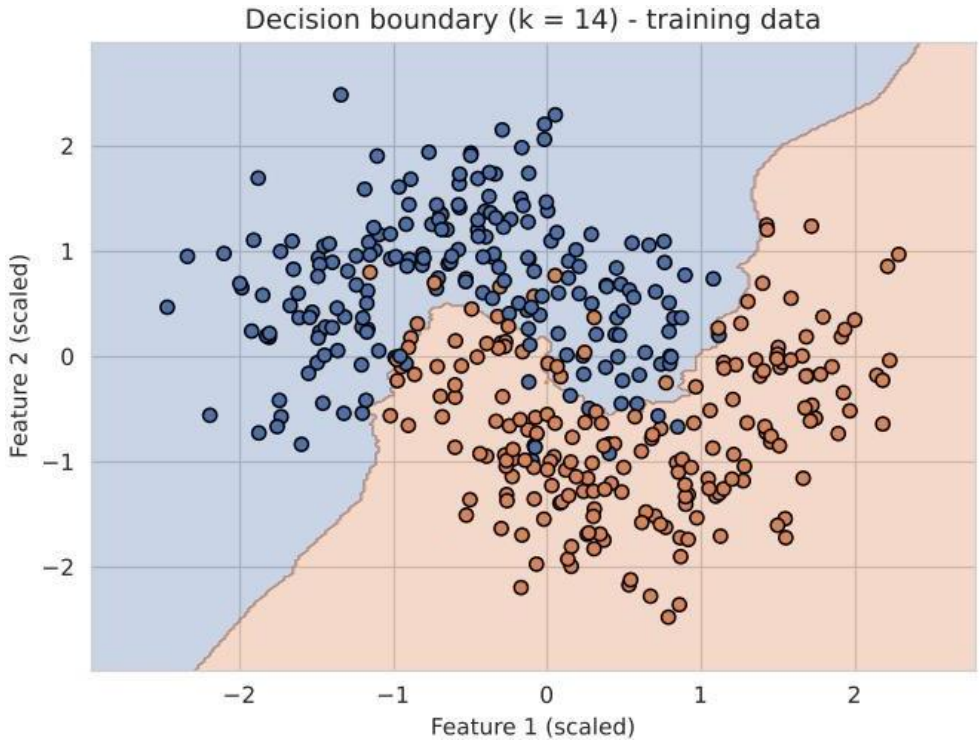


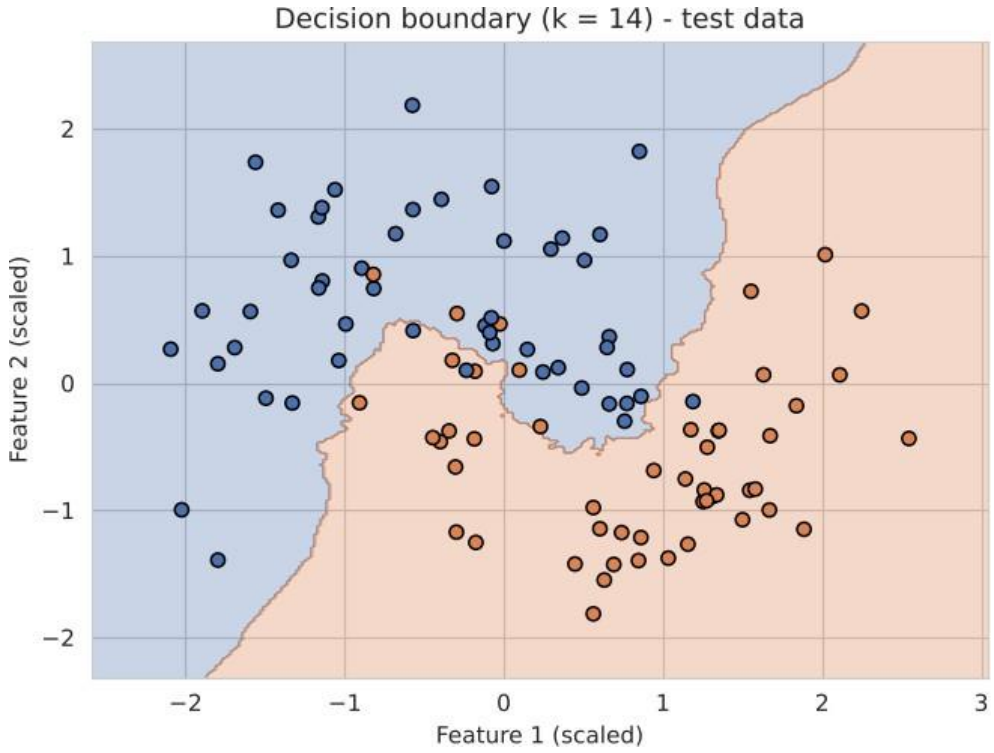

```
def plot_decision_boundary(model, X_set, y_set, title):
    x_min, x_max = X_set[:, 0].min() - 0.5, X_set[:, 0].max() + 0.5
    y_min, y_max = X_set[:, 1].min() - 0.5, X_set[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300), np.linspace(y_min, y_max, 300))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=ListedColormap(["#4c72b0", "#dd8452"]))
    plt.scatter(X_set[:, 0], X_set[:, 1], c=y_set, edgecolor="k",
                cmap=ListedColormap(["#4c72b0", "#dd8452"]))
    plt.xlabel("Feature 1 (scaled)"); plt.ylabel("Feature 2 (scaled)")
    plt.title(title)

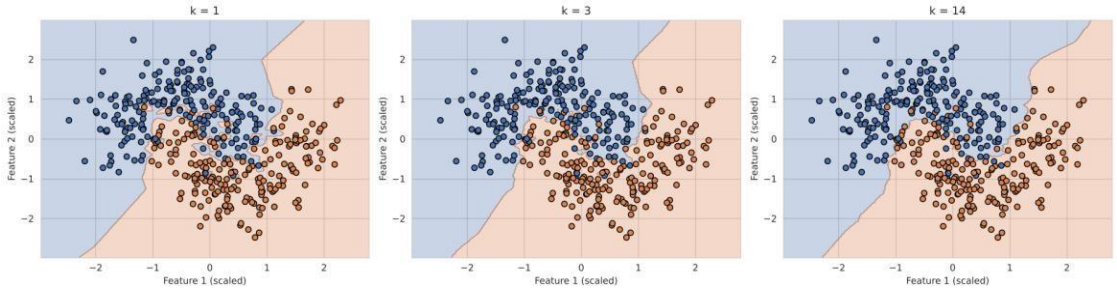
plt.figure(figsize=(7, 5))
plot_decision_boundary(knn_best, X_train, y_train, f"Decision boundary (k = {best_k}) - training data")
plt.show()

plt.figure(figsize=(7, 5))
plot_decision_boundary(knn_best, X_test, y_test, f"Decision boundary (k = {best_k}) - test data")
plt.show()
```






```
# Optional: compare boundaries for small k, k=3 and best k
fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
for ax, k in zip(axes, [1, 3, best_k]):
    plt.sca(ax)
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    plot_decision_boundary(m, X_train, y_train, f"k = {k}")
plt.tight_layout()
plt.show()
```



Practical - 7

XGBoost Classification

Aim:

To implement the XGBoost algorithm for classification using the Breast Cancer dataset and evaluate its performance using accuracy, a classification report, and graphical visualization.

Theory

XGBoost (Extreme Gradient Boosting) is a powerful supervised machine learning algorithm used for classification and regression tasks. It is an optimized implementation of the gradient boosting technique that combines multiple decision trees to improve prediction accuracy and model performance.

In this practical, XGBoost is applied to the Breast Cancer dataset to classify tumors as malignant (cancerous) or benign (non-cancerous). The model is evaluated using accuracy, a classification report, and graphical visualizations.

1. XGBoost Algorithm

XGBoost is an ensemble learning method based on boosting. It builds decision trees sequentially, where each new tree attempts to correct the errors made by the previous trees.

The main features of XGBoost are:

- Gradient Boosting: Improves predictions by learning from previous errors.
- Regularization: Helps control model complexity and reduce overfitting.
- Parallel Processing: Uses computational optimizations for efficient training.
- Missing Value Handling: Can learn appropriate directions for missing values during tree splitting.
- Feature Importance: Helps identify features that contribute to predictions.

2. Breast Cancer Dataset

The Breast Cancer dataset contains numerical measurements of cell nuclei obtained from breast mass samples. It includes features such as:

- Radius
- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity

The target variable contains two classes:

- Malignant: Cancerous tumor.
- Benign: Non-cancerous tumor.

The dataset is used to train the XGBoost model and evaluate its ability to classify tumors correctly.

3. Working of XGBoost

The basic working process of XGBoost is:

1. Initialize the model with an initial prediction.
2. Calculate the errors or residual information from the current predictions.
3. Build a new decision tree to improve the predictions.
4. Add the new tree's predictions to the existing model using a learning rate.
5. Repeat the process for a specified number of boosting rounds.
6. Generate the final class prediction using the combined output of all trees.

4. Performance Evaluation

The XGBoost model is evaluated using the following metrics.

Accuracy: Measures the proportion of correct predictions.

Precision: Measures how many predicted positive cases are actually positive.

Recall: Measures how many actual positive cases are correctly identified.

F1-score: Represents the harmonic mean of precision and recall.

A classification report presents precision, recall, F1-score, and support for each class, along with summary averages.

5. Graphical Visualization

Graphical visualization helps interpret the performance and behavior of the trained model. Common visualizations include:

- Confusion Matrix: Shows correct and incorrect predictions for each class.
- Feature Importance Plot: Displays the features that contribute most to the model's predictions.
- Class Distribution Chart: Shows the distribution of malignant and benign samples.

These visualizations make it easier to understand classification results and the contribution of different features.

Code and Output:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
```

```
# Load dataset
data = load_breast_cancer()
X = data.data
```

```
y = data.target

print("Dataset loaded successfully.")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Target classes:", data.target_names)
```

Output:

```
Dataset loaded successfully.
Number of samples: 569
Number of features: 30
Target classes: ['malignant' 'benign']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output:

```
Training samples: 455
Testing samples: 114
```

```
# Create XGBoost model
model = XGBClassifier(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.1,
    eval_metric="logloss",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

print("XGBoost model trained successfully.")
```

Output:

```
XGBoost model trained successfully.
```

```
# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred) * 100

print("XGBoost Accuracy:", round(accuracy, 2), "%")

print("\nClassification Report:")
print(
    classification_report(
```

```
        y_test,
        y_pred,
        target_names=data.target_names
    )
)
```

Output:

```
XGBoost Accuracy: 94.74 %

Classification Report:
              precision    recall  f1-score   support

   malignant       0.95      0.90      0.93        42
      benign       0.95      0.97      0.96        72

    accuracy                           0.95       114
   macro avg       0.95      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114
```

```
# Practical 7: XGBoost Confusion Matrix

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Calculate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

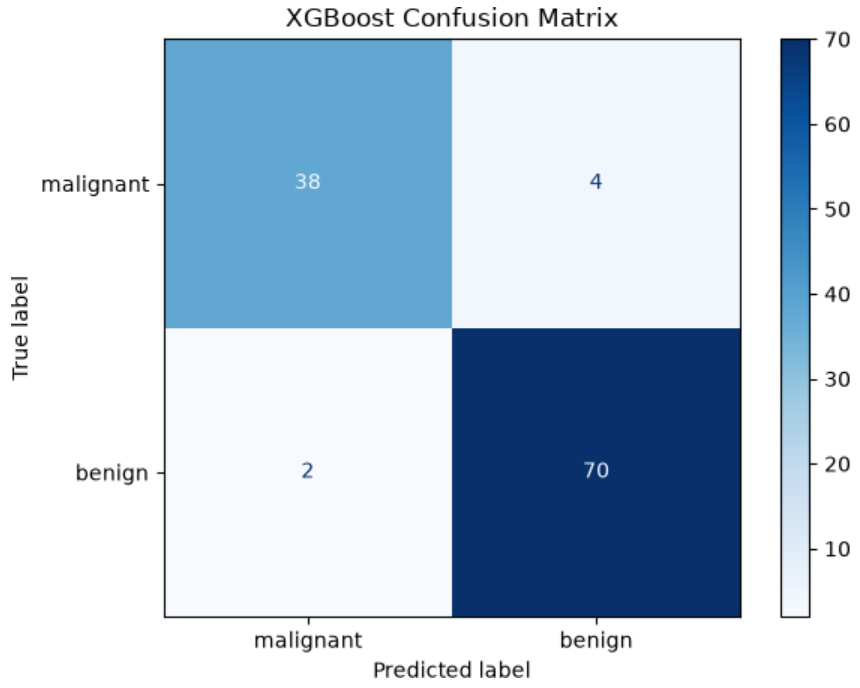
# Plot confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)

disp.plot(cmap="Blues", values_format="d")

plt.title("XGBoost Confusion Matrix")
plt.tight_layout()
plt.show()
```

Output:

```
Confusion Matrix:
[[38  4]
 [ 2 70]]
```




```
# Practical 7: Precision, Recall, and F1-Score Graph

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate evaluation metrics
precision = precision_score(y_test, y_pred, average="weighted")
recall = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")

# Metric names and values
metrics = ["Precision", "Recall", "F1-Score"]
scores = [precision, recall, f1]

# Plot bar graph
plt.figure(figsize=(8, 5))

bars = plt.bar(
    metrics,
    scores,
    color=["skyblue", "lightgreen", "orange"],
    edgecolor="black",
    width=0.5
)

# Display values above bars
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.01,
        f"{height:.3f}",
        ha="center",
        va="bottom",
```

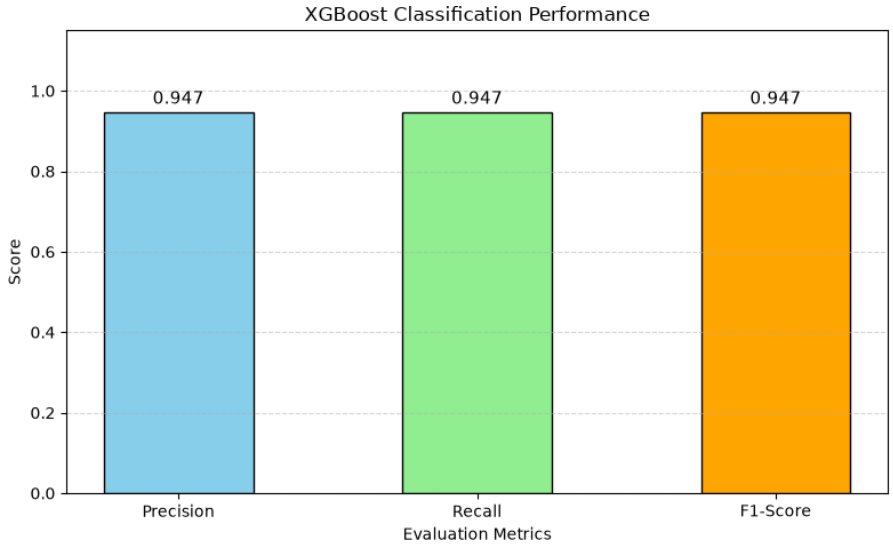
```
        fontsize=11
    )

plt.title("XGBoost Classification Performance")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.ylim(0, 1.15)
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

# Print metric values
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
```

Output:




```
Precision: 0.9474
Recall: 0.9474
F1-Score: 0.9471
```

```
# Display model accuracy using a bar graph
plt.figure(figsize=(7, 5))

bars = plt.bar(
    ["XGBoost"],
    [accuracy],
    color="skyblue",
    edgecolor="black",
    width=0.5
)

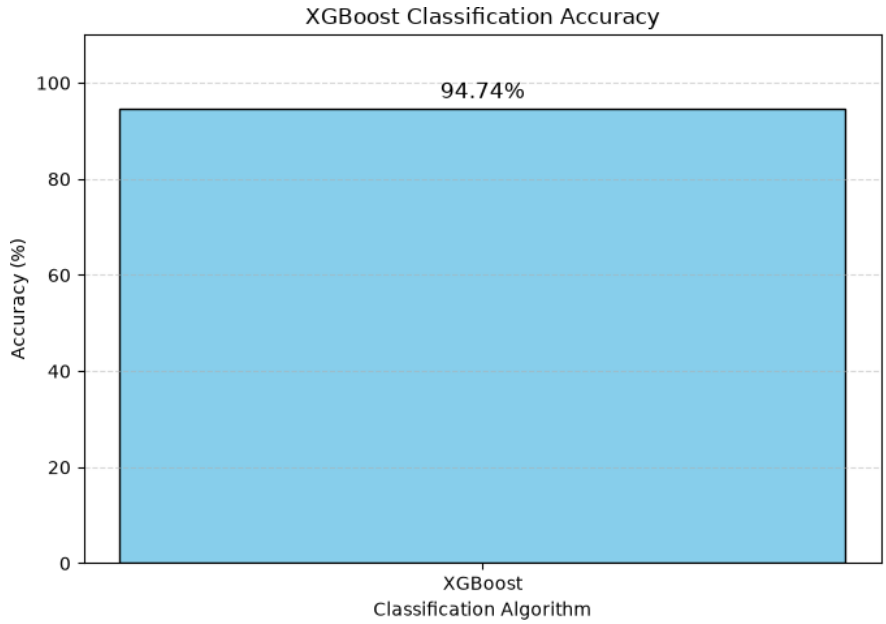
# Display accuracy value above the bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"{height:.2f}%",
```

```
        ha="center",
        va="bottom",
        fontsize=12
    )

plt.title("XGBoost Classification Accuracy")
plt.xlabel("Classification Algorithm")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 110)
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

Output:



Practical No. 8

Classification using SVM on any user defined Dataset

Aim:

To implement Support Vector Machine (SVM) classification on a user-defined dataset.

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the best decision boundary to separate data points belonging to different classes. In this practical, SVM is implemented on a user-defined dataset to train a classification model and evaluate its performance.

1. Support Vector Machine (SVM)

SVM finds an optimal hyperplane that separates different classes while maximizing the margin between them. The margin is the distance between the hyperplane and the nearest data points from each class.

The nearest data points are called support vectors, and they play an important role in determining the decision boundary.

2. Types of SVM

- Linear SVM: Used when data can be separated using a straight line or linear hyperplane.
- Nonlinear SVM: Used when data cannot be separated linearly.
- Kernel SVM: Uses kernel functions to handle nonlinear classification problems.

Common kernel functions include Linear, Polynomial, and RBF (Radial Basis Function).

3. Working of SVM

1. Load or create a user-defined dataset.
2. Separate the input features and target classes.
3. Split the dataset into training and testing sets.
4. Scale the features if required.
5. Train the SVM classifier using the training data.
6. Predict class labels for the test data.
7. Evaluate the model using accuracy and a confusion matrix.

4. Performance Evaluation

The performance of the SVM model can be evaluated using the following metrics:

- Accuracy: Measures the percentage of correct predictions.
- Precision: Measures the correctness of positive predictions.
- Recall: Measures how many actual positive cases are correctly identified.
- F1-score: Represents the balance between precision and recall.
- Confusion Matrix: Displays correct and incorrect predictions for each class.

5. Advantages of SVM

- Effective for high-dimensional datasets.
- Can handle linear and nonlinear classification.
- Uses support vectors to determine the decision boundary.
- Can perform well when classes have a clear margin of separation.

6. Applications of SVM

- Image and handwriting recognition.
- Email spam detection.
- Text classification.
- Medical diagnosis.
- Face recognition and pattern detection.

Code and Output:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay, precision_score, recall_score, f1_score
)
```

```
# Create 100 synthetic user-defined student records
data = {
    "Study_Hours": [
        1,2,3,4,5,6,7,8,2,6,3,9,1,4,5,7,8,3,6,9,
        2,4,6,8,1,5,7,9,3,4,6,8,2,5,7,9,1,3,5,8,
        4,6,7,2,9,3,5,8,1,6,4,7,9,2,5,8,3,6,1,7,
        9,4,5,2,8,6,3,7,1,9,5,4,8,2,6,7,3,9,1,5,
        6,8,4,7,2,9,3,5,1,8,6,4,7,2,9,5,3,8,6,7
    ],
    "Attendance": [
        40,45,50,60,65,70,80,90,55,75,48,95,35,58,68,82,88,52,73,96,
        42,62,72,92,38,66,78,94,49,57,76,89,46,64,81,98,33,54,69,91,
        61,74,85,44,97,51,67,87,39,71,59,83,99,47,63,93,53,77,36,84,
        96,56,70,43,90,72,50,86,37,98,65,60,88,45,75,80,52,95,34,68,
        73,92,58,82,48,99,55,66,41,89,76,62,85,46,97,69,51,91,74,87
    ],
    "Result": [
        0,0,0,0,1,1,1,1,0,1,0,1,0,0,1,1,1,0,1,1,
        0,0,1,1,0,1,1,1,0,0,1,1,0,1,1,1,0,0,1,1,
        0,1,1,0,1,0,1,1,0,1,0,1,1,0,1,1,0,1,0,1,
        1,0,1,0,1,1,0,1,0,1,1,0,1,0,1,1,0,1,0,1,
        1,1,0,1,0,1,0,1,0,1,1,0,1,0,1,1,0,1,1,1
    ]
}
```

```
df = pd.DataFrame(data)
print("Dataset (first 10 records):")
```

```
print(df.head(10))
print("\nTotal records:", len(df))
print("\nClass counts (0 = Fail, 1 = Pass):")
print(df["Result"].value_counts().sort_index())
```

Output:

```
Dataset (first 10 records):
   Study_Hours  Attendance  Result
0            1          40       0
1            2          45       0
2            3          50       0
3            4          60       0
4            5          65       1
5            6          70       1
6            7          80       1
7            8          90       1
8            2          55       0
9            6          75       1

Total records: 100

Class counts (0 = Fail, 1 = Pass):
Result
0    40
1    60
Name: count, dtype: int64
```

```
# Separate features and target
X = df[["Study_Hours", "Attendance"]]
y = df["Result"]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

Output:

```
Training records: 75
Testing records: 25
```

```
# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#  Train  SVM  classifier
model = SVC(kernel="linear")
model.fit(X_train_scaled, y_train)
print("SVM model trained successfully.")
```

Output:

```
SVM model trained successfully.
```

```
# Predict and evaluate
```

```
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred) * 100

print("Actual Results:", list(y_test))
print("Predicted Results:", list(y_pred))
print("\nSVM Accuracy:", round(accuracy, 2), "%")
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=["Fail", "Pass"], zero_division=0
))
```

Output:

```
Actual Results: [1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1]
Predicted Results: [np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1),
np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1),
np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0),
np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(1)]

SVM Accuracy: 100.0 %

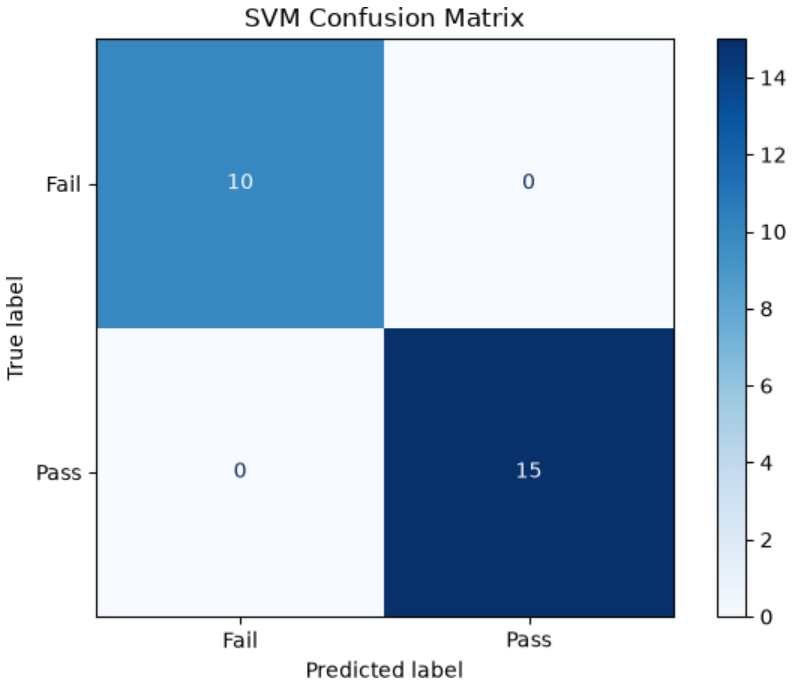
Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00        10
        Pass       1.00      1.00      1.00        15

    accuracy                           1.00        25
   macro avg       1.00      1.00      1.00        25
weighted avg       1.00      1.00      1.00        25
```

```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=["Fail", "Pass"]
)
disp.plot(cmap="Blues", values_format="d")
plt.title("SVM Confusion Matrix")
plt.tight_layout()
plt.show()
```

Output:




```
# Precision, Recall, and F1-score graph (weighted averages)
precision = precision_score(y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

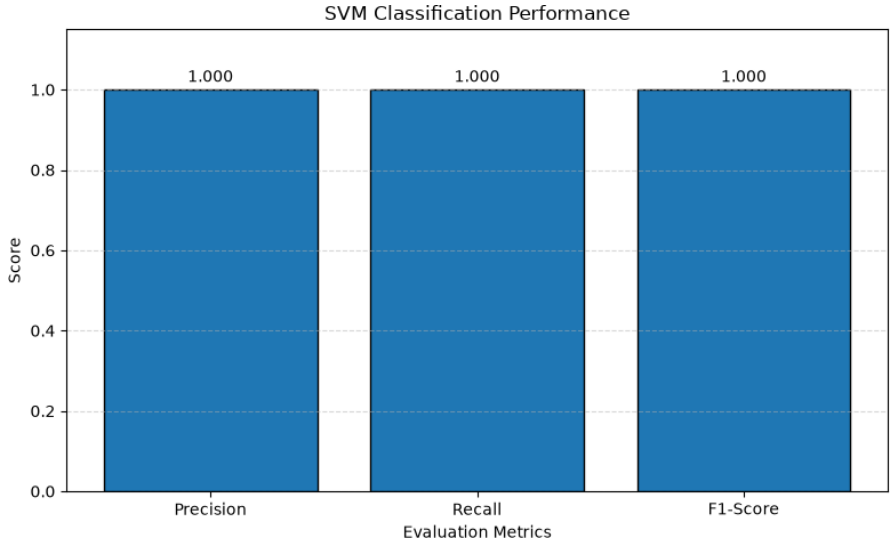
metric_names = ["Precision", "Recall", "F1-Score"]
metric_values = [precision, recall, f1]

plt.figure(figsize=(8, 5))
bars = plt.bar(metric_names, metric_values, edgecolor="black")
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height + 0.01,
             f"{height:.3f}", ha="center", va="bottom")

plt.title("SVM Classification Performance")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.ylim(0, 1.15)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
```

Output:




```
Precision: 1.0
Recall: 1.0
F1-Score: 1.0
```

Practical No.9

K-Fold Cross-Validation on a Regression Model

Aim:

Perform k-Fold Cross-Validation on a regression model to evaluate its performance on different subsets of the data.

Theory

K-Fold Cross-Validation is a model evaluation technique used in machine learning to estimate how well a model performs on unseen data. It divides the dataset into \(K\) approximately equal-sized subsets called folds. In this practical, K-Fold Cross-Validation is applied to a regression model to evaluate its performance across different subsets of the data.

1. K-Fold Cross-Validation

In K-Fold Cross-Validation, the dataset is divided into \(K\) folds. The model is trained \(K\) times, using \(K-1\) folds for training and the remaining fold for validation in each iteration.

For example, in 5-fold cross-validation, the model is trained five times, and each fold is used once for validation.

2. Working of K-Fold Cross-Validation

1. Load the dataset and select the input features and target variable.
2. Divide the dataset into K folds.
3. Select one fold for validation and use the remaining folds for training.
4. Train the regression model on the training data.
5. Evaluate the model on the validation fold.
6. Repeat the process until every fold has been used for validation.
7. Calculate the average performance score across all folds.

3. Performance Evaluation Metrics

Regression models can be evaluated using the following metrics:

- Mean Absolute Error (MAE): Measures the average absolute difference between actual and predicted values.
- Mean Squared Error (MSE): Measures the average squared prediction error.
- Root Mean Squared Error (RMSE): Represents the square root of MSE.
- R^2 Score: Measures how well the model explains the variation in the target variable.

4. Advantages of K-Fold Cross-Validation

- Provides a more reliable estimate of model performance.
- Uses different subsets of data for training and validation.
- Helps identify overfitting and underfitting.

- Reduces dependence on a single train-test split.
- Supports comparison between different regression models.

5. Applications

- House price prediction.
- Sales and revenue forecasting.
- Temperature prediction.
- Demand estimation.
- Evaluating machine learning regression models.

Code and Output:

```
# Import libraries
import pandas as pd
import numpy as np

from pathlib import Path
from sklearn.model_selection import KFold, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import make_scorer, mean_squared_error

import warnings
warnings.filterwarnings("ignore")
```

```
# Put Mall_Customers.csv in the same folder as this notebook.
# The second path supports the original uploaded-file environment.
csv_path = Path("Mall_Customers.csv")
if not csv_path.exists():
    csv_path = Path("/mnt/data/Mall_Customers.csv")

df = pd.read_csv(csv_path)
print("Dataset shape:", df.shape)
display(df.head())
print("\\nColumns:", df.columns.tolist())
print("\\nMissing values per column:\\n", df.isnull().sum())
```

Output:

```
Dataset shape: (200, 5)
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
\nColumns: ['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']
\nMissing values per column:\n CustomerID                0
Gender                    0
```

```
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64
```

```
# Features used to predict the spending score
feature_columns = ["Age", "Annual Income (k$)", "Gender"]
target_column = "Spending Score (1-100)"

X = df[feature_columns]
y = df[target_column]

numeric_features = ["Age", "Annual Income (k$)"]
categorical_features = ["Gender"]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

Output:

```
Feature shape: (200, 3)
Target shape: (200,)
```

```
# Preprocess numeric and categorical columns separately.
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

model
```

Output:

```
Pipeline(steps=[('preprocessor',
                 ColumnTransformer(transformers=[('numeric',
                                                  Pipeline(steps=[('imputer',

SimpleImputer(strategy='median')),
                                                                  ('scaler',
                                                                   StandardScaler())]),
                                                  ['Age',
                                                   'Annual Income (k$)']),
                                                 ('categorical',
                                                 Pipeline(steps=[('imputer',
```

```
SimpleImputer(strategy='most_frequent')),
                                                                    ('onehot',

OneHotEncoder(handle_unknown='ignore'))]),
                                                  ['Gender'])])),
                ('regressor', LinearRegression())])
```

```
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)

scoring = {
    "r2": "r2",
    "mae": "neg_mean_absolute_error",
    "mse": "neg_mean_squared_error"
}

cv_results = cross_validate(
    model,
    X,
    y,
    cv=kf,
    scoring=scoring,
    return_train_score=False
)

results = pd.DataFrame({
    "Fold": np.arange(1, k + 1),
    "R²": cv_results["test_r2"],
    "MAE": -cv_results["test_mae"],
    "RMSE": np.sqrt(-cv_results["test_mse"])
})

display(results.round(3))
```

Output:

```
   Fold     R²     MAE    RMSE
0     1  0.025  18.151  21.924
1     2  0.090  20.993  25.028
2     3  0.014  23.900  28.025
3     4  0.172  20.027  23.258
4     5 -0.041  20.659  25.144
```

```
summary = pd.DataFrame({
    "Metric": ["R²", "MAE", "RMSE"],
    "Mean": [
        results["R²"].mean(),
        results["MAE"].mean(),
        results["RMSE"].mean()
    ],
    "Standard Deviation": [
        results["R²"].std(),
        results["MAE"].std(),
        results["RMSE"].std()
    ]
})
```

```
display(summary.round(3))

print(f"Average R²:   {results['R²'].mean():.3f}")
print(f"Average MAE:  {results['MAE'].mean():.3f} spending-score points")
print(f"Average RMSE: {results['RMSE'].mean():.3f} spending-score points")
```

Output:

```
  Metric    Mean  Standard Deviation
0     R²   0.052               0.081
1    MAE  20.746               2.078
2   RMSE  24.676               2.299
```

```
Average R²:   0.052
Average MAE:  20.746 spending-score points
Average RMSE: 24.676 spending-score points
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

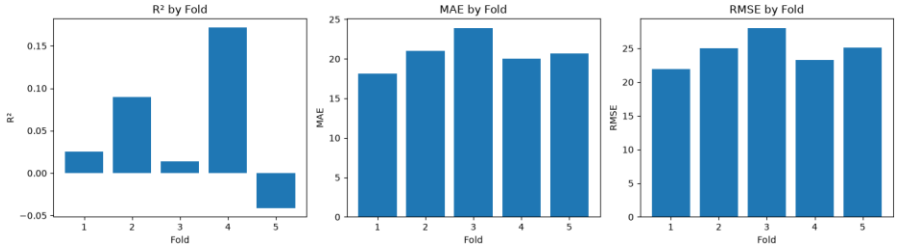
axes[0].bar(results["Fold"], results["R²"])
axes[0].set_title("R² by Fold")
axes[0].set_xlabel("Fold")
axes[0].set_ylabel("R²")

axes[1].bar(results["Fold"], results["MAE"])
axes[1].set_title("MAE by Fold")
axes[1].set_xlabel("Fold")
axes[1].set_ylabel("MAE")

axes[2].bar(results["Fold"], results["RMSE"])
axes[2].set_title("RMSE by Fold")
axes[2].set_xlabel("Fold")
axes[2].set_ylabel("RMSE")

plt.tight_layout()
plt.show()
```

Output:



Practical No.10

DBSCAN algorithm

Aim:

Implement DBSCAN algorithm.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised machine learning algorithm used to group data points based on their density. It identifies clusters of closely packed data points and detects outliers or noise points that do not belong to any cluster.

Unlike K-Means, DBSCAN does not require the number of clusters to be specified in advance and can identify clusters of different shapes.

1. Important Parameters of DBSCAN
   - Epsilon (eps): The maximum distance between two points for them to be considered neighbors.
   - MinPts: The minimum number of points required within the epsilon neighborhood to form a dense region.
2. Types of Points in DBSCAN
   - Core Point: A point that has at least MinPts points within its epsilon neighborhood.
   - Border Point: A point that has fewer neighbors than MinPts but lies within the neighborhood of a core point.
   - Noise Point: A point that does not belong to any cluster and is considered an outlier.
3. Working of DBSCAN
   1. Select the values of eps and min_samples.
   2. Choose an unvisited data point.
   3. Find all neighboring points within the epsilon distance.
   4. If enough neighbors exist, form a new cluster.
   5. Expand the cluster by adding reachable neighboring points.
   6. Mark points that do not belong to any cluster as noise.
   7. Repeat the process until all points have been visited.
4. Advantages of DBSCAN
   - Does not require the number of clusters beforehand.
   - Detects outliers and noise points.
   - Can identify clusters of irregular shapes.
   - Works well for spatial and density-based data.
5. Applications of DBSCAN
   - Customer segmentation.
   - Geographic and spatial data analysis.
   - Anomaly and outlier detection.

- Image processing.
- Identifying suspicious transaction patterns.

Code and Output:

```
# If needed, install dependencies by running:
# %pip install pandas numpy matplotlib scikit-learn seaborn

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, silhouette_score

RANDOM_STATE = 42
sns.set_theme(style="whitegrid")
```

```
# Load the CSV. Keep diabetes.csv in the same folder as this notebook.
df = pd.read_csv("diabetes.csv")

print("Dataset shape:", df.shape)
display(df.head())
display(df.describe().T)
print("\\nMissing values  per column:")
display(df.isna().sum())
```

Output:

```
Dataset shape: (768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

	count	mean	std	min	25%	\
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	
Glucose	768.0	120.894531	31.972618	0.000	99.00000	
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	
Insulin	768.0	79.799479	115.244002	0.000	0.00000	

```
BMI                       768.0   31.992578    7.884160   0.000  27.30000
DiabetesPedigreeFunction  768.0    0.471876    0.331329   0.078   0.24375
Age                       768.0   33.240885   11.760232  21.000  24.00000
Outcome                   768.0    0.348958    0.476951   0.000   0.00000

                              50%        75%     max
Pregnancies                3.0000    6.00000   17.00
Glucose                  117.0000  140.25000  199.00
BloodPressure             72.0000   80.00000  122.00
SkinThickness             23.0000   32.00000   99.00
Insulin                   30.5000  127.25000  846.00
BMI                       32.0000   36.60000   67.10
DiabetesPedigreeFunction   0.3725    0.62625    2.42
Age                       29.0000   41.00000   81.00
Outcome                    0.0000    1.00000    1.00
```

```
\nMissing values per column:
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
target_col = "Outcome"
zero_as_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]

data = df.copy()
for col in zero_as_missing:
    if col in data.columns:
        data.loc[data[col] == 0, col] = np.nan

X = data.drop(columns=[target_col], errors="ignore")
y = df[target_col] if target_col in df.columns else None

# Ensure only numeric features are used
X = X.select_dtypes(include=[np.number])

imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

X_imputed = imputer.fit_transform(X)
X_scaled = scaler.fit_transform(X_imputed)

print("Features used:", list(X.columns))
print("Rows:", X_scaled.shape[0], "| Features:", X_scaled.shape[1])
print("Remaining missing values after imputation:", np.isnan(X_imputed).sum())
```

Output:

```
Features used: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age']
Rows: 768 | Features: 8
```

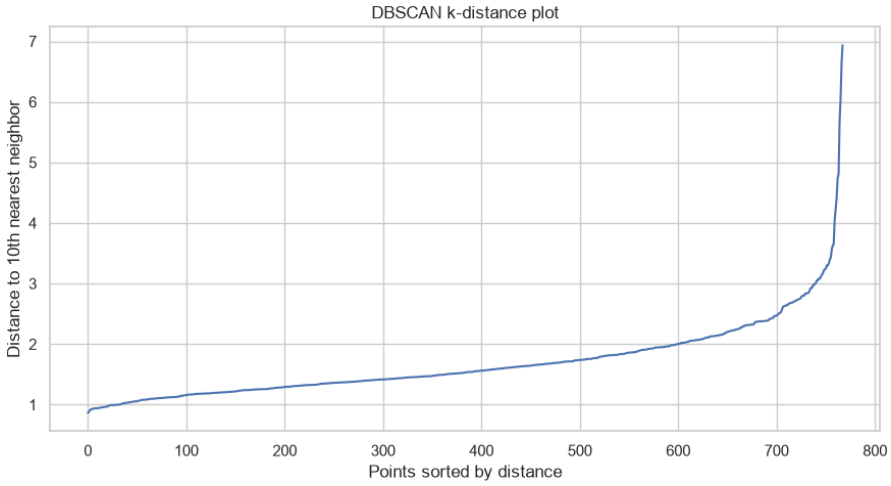
```
Remaining missing values after imputation: 0
```

```
min_samples = 10

neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)
k_distances = np.sort(distances[:, -1])

plt.figure(figsize=(9, 5))
plt.plot(k_distances)
plt.xlabel("Points sorted by distance")
plt.ylabel(f"Distance to {min_samples}th nearest neighbor")
plt.title("DBSCAN k-distance plot")
plt.tight_layout()
plt.show()
```

Output:




```
# Tune these values after reviewing the k-distance plot.
eps = 1.5
min_samples = 10

dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels = dbscan.fit_predict(X_scaled)

df_results = df.copy()
df_results["Cluster"] = labels

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = int(np.sum(labels == -1))
noise_pct = 100 * n_noise / len(labels)

print(f"eps={eps}, min_samples={min_samples}")
print("Number of clusters (excluding noise):", n_clusters)
print(f"Noise points: {n_noise} ({noise_pct:.1f}%)")
display(df_results["Cluster"].value_counts().sort_index().rename("Count").to_frame())
display(df_results.head())
```

Output:

```
eps=1.5, min_samples=10
Number of clusters (excluding noise): 1
```

```
Noise points: 231 (30.1%)
```

```
         Count
Cluster
-1         231
 0         537
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin    BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome  Cluster
0                     0.627   50        1        0
1                     0.351   31        0        0
2                     0.672   32        1       -1
3                     0.167   21        0        0
4                     2.288   33        1       -1
```

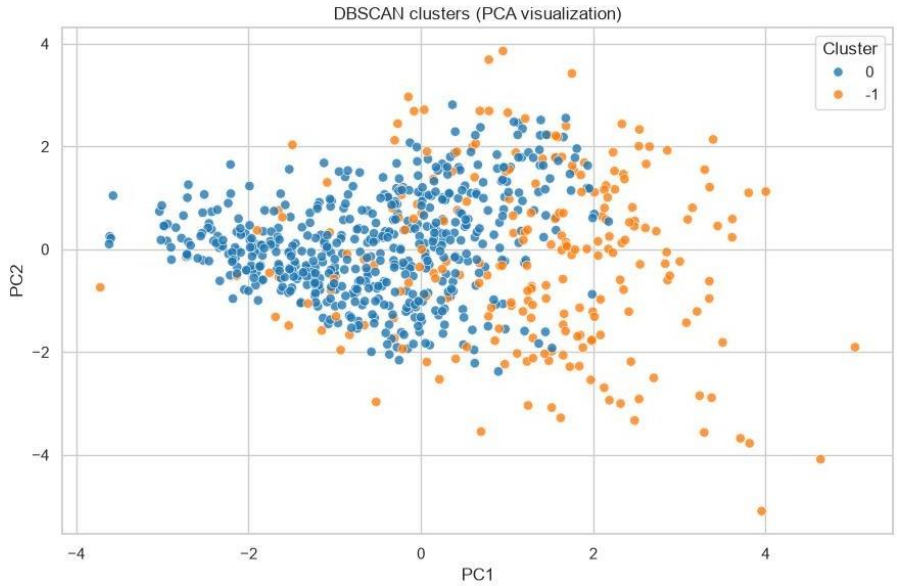
```
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_2d = pca.fit_transform(X_scaled)

plot_df = pd.DataFrame({
    "PC1": X_2d[:, 0],
    "PC2": X_2d[:, 1],
    "Cluster": labels.astype(str)
})

plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=plot_df, x="PC1", y="PC2",
    hue="Cluster", palette="tab10", s=45, alpha=0.8
)
plt.title("DBSCAN clusters (PCA visualization)")
plt.tight_layout()
plt.show()

print("Variance explained by first two PCs:",
      f"{pca.explained_variance_ratio_.sum():.1%}")
```

Output:




```
Variance explained by first two PCs: 47.2%
```

```
feature_cols = [c for c in X.columns if c in df_results.columns]
cluster_summary = df_results.groupby("Cluster")[feature_cols].median().round(2)
cluster_sizes = df_results["Cluster"].value_counts().sort_index().rename("Count")
display(cluster_sizes.to_frame().join(cluster_summary))
```

Output:

```
         Count  Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  \
Cluster
-1         231          4.0    142.0           76.0           30.0    125.0
 0         537          3.0    110.0           70.0           20.0      0.0

           BMI  DiabetesPedigreeFunction   Age
Cluster
-1        35.0                      0.60  38.0
 0        30.8                      0.32  27.0
```

```
if y is not None:
    print("Cluster vs Outcome crosstab (counts):")
    display(pd.crosstab(df_results["Cluster"], y, margins=True))

    ari = adjusted_rand_score(y, labels)
    print(f"Adjusted Rand Index vs Outcome: {ari:.4f}")

    non_noise = labels != -1
    unique_non_noise = set(labels[non_noise])
    if non_noise.sum() > 1 and len(unique_non_noise) >= 2:
        sil = silhouette_score(X_scaled[non_noise], labels[non_noise])
        print(f"Silhouette score on non-noise points: {sil:.4f}")
    else:
        print("Silhouette score skipped: fewer than 2 non-noise clusters.")
else:
    print("No Outcome column found; skipping target comparison.")
```

Output:

```
Cluster vs Outcome crosstab (counts):
```

```
Outcome    0    1  All
Cluster
-1       102  129  231
0        398  139  537
All      500  268  768
```

```
Adjusted Rand Index vs Outcome: 0.1251
Silhouette score skipped: fewer than 2 non-noise clusters.
```

```
eps_values = [0.8, 1.0, 1.2, 1.5, 1.8, 2.0]
sensitivity_rows = []

for candidate_eps in eps_values:
    candidate_labels = DBSCAN(
        eps=candidate_eps, min_samples=min_samples
    ).fit_predict(X_scaled)
    candidate_clusters = len(set(candidate_labels)) - (1 if -1 in candidate_labels else 0)
    candidate_noise = int(np.sum(candidate_labels == -1))
    sensitivity_rows.append({
        "eps": candidate_eps,
        "clusters_excluding_noise": candidate_clusters,
        "noise_points": candidate_noise,
        "noise_percent": round(100 * candidate_noise / len(candidate_labels), 1)
    })

sensitivity = pd.DataFrame(sensitivity_rows)
display(sensitivity)
```

Output:

```
   eps  clusters_excluding_noise  noise_points  noise_percent
0  0.8                         0           768          100.0
1  1.0                         1           668           87.0
2  1.2                         1           469           61.1
3  1.5                         1           231           30.1
4  1.8                         1           115           15.0
5  2.0                         1            67            8.7
```